

Shibboleth SSO 検証環境 構築手順書 (純 Windows 版)

第 0.26 版

2026-07-16

0. 改訂履歴

版	日付	変更概要	備考
0.1	2026-07-05	純 Windows 版として新規作成。\$1～\$4 (目的・前提・全体アーキテクチャ・パラメータ・ロードマップ) を記載	WSL 版 v0.15 を土台に、顧客 PLM 検証環境の再現用として起こす
0.2	2026-07-05	フェーズ 1 (検証環境の初期設定: 前提確認・時刻同期・着手前チェックポイント) を追記。推奨アカウント値 (わかりやすさ優先) を \$3 に追記	入れ子仮想化は不要のため簡素化
0.3	2026-07-05	フェーズ 2 hosts・内部 CA・idp/sp サーバ証明書(a)の発行・PDF 化・信頼登録) を追記。証明書作成は Git for Windows 同梱の openssl を使用	ネットワーク方式選定は不要 (同一 Windows・127.0.0.1)
0.4	2026-07-05	フェーズ 3 ApacheDS 導入・パーティション作成・ou=people・テストユーザー 01PLM01/02・検索バインド idp-reader) を追記	LDAP は ApacheDS。GUI (Directory Studio) + LDIF
0.5	2026-07-08	フェーズ 3 の実機知見を反映 Java 前倒し / zip 優先方針 / C: 統一 / ApacheDS は LDAP ポート 10389・設定は ou=config 方式で config.ldif 無し - 既定パーティション dc=example, dc=com 流用 / Directory Studio は exe 起動 / userPassword は SSHA 自動ハッシュ)。\$6.4.1「証明書コマンドの解説」を追記	実機検証の反映と解説の追加
0.6	2026-07-08	フェーズ 4 OpenJDK は \$5.6 で導入済み。Tomcat 10.1 の zip 展開・環境変数・service.bat によるサービス化・8080 起動確認) を追記	Tomcat は zip + service.bat

版	日付	変更概要	備考
0.7	2026-07-08	フェーズ5 Shibboleth IdP 5 : zip 展開 + install.bat 導入・context フラグメント配置・ldap.properties ⑩389/dc=example,dc=com/idp-reader)・emailAddress NameID の元 mail・起動確認)を追記。フェーズ4のサービス表示名を実機値「Apache Tomcat 10.1 Tomcat10」に補足	本構成の山場
0.8	2026-07-09	フェーズ5 実機反映 : \$9.3 を訂正し JSTL (API + Glassfish 実装の2 jar) 追加 + build.bat 再ビルドを必須手順に (未追加だと /idp/profile/status が ClassNotFoundException: jakarta.servlet.jsp.jstl.core.Config-ServletException) 。 \$9.4 に trustCertificates /trustStore のコメントアウト、saml-nameid の bean 重複回避を追記	JSTL は必須だった
0.9	2026-07-09	フェーズ6 Tomcat 直 HTTPS 8443 公開 : idp.pfx を conf に配置し server.xml に SSLHostConfig + Certificate の 8443 コネクタを追加、8080 は localhost 限定、鍵マーク確認)を追記。Apache 前段は不要	WSL 版の Apache + mirrored を Tomcat コネクタ1つで代替
0.10	2026-07-09	フェーズ7 IIS 導入・sp.pfx 取込・既定サイトに 443/HTTPS バインド・確認用 whoami.asp) を追記。sp.pfx は C:\lab\ca から取込。UAC 承認モード差異・ASP 文字コードの補足を反映	WSL 版フェーズ7 を流用 (mirrored 不要)
0.11	2026-07-09	フェーズ8 Shibboleth SP : MSI 導入・IIS ネイティブモジュール確認・shibboleth2.xml 編集 (SAPI/RequestMapper/entityID /SSO) ・keygen で鍵(b)・サイト全体保護)を追記	WSL 版フェーズ8 を流用
0.12	2026-07-09	フェーズ8 実機反映 (MSI 表記「Configure IIS support」/サービス表示名「Shibboleth Daemon	:8443 補正が不要になり簡素化

版	日付	変更概要	備考
0.13	2026-07-10	(Default)」／shibd.exe は sbin64 または sbin／使用インストーラ版数一覧)。フェーズ9 (メタデータ相互登録・初回 SSO・: 8443 補正は不要) を追記 フェーズ9 実機反映: \$13.1 を訂正し :8443 補正は必要 {instal.l.bat 生成のメタデータはエンドポイントがポートなしのため) に。フェーズ10 {emailAddress 形式 NameID を mail から生成し REMOTE_USER に載せる: 目標 01PLM01@plm-lab.local) を追記	uid/unspecified-mail/emailAddress
0.14	2026-07-11	フェーズ10 実機反映: \$14.1 を「既存 mail テンプレートのドメインだけ変更 (二重定義しない。Duplicate Definition 'mail' WARN の回避)」に整理。\$14.5 に partner-metadata.xml (不在なら該当 MetadataProvider をコメントアウト) とログの時刻フィルタ確認を追記。REMOTE_USER=[01PLM01@plm-lab.local] を達成 (目標到達)	属性連携 完了
0.15	2026-07-11	フェーズ11 (結合テスト: 通し確認・01PLM02 再現・セッション確認・ログアウト対象外・ログの読み方・再起動堅牢性 (Windows サービス自動起動でオンデマンド問題なし)・問題早見表・本番展開メモ) を追記。全 11 フェーズ完了	純 Windows 版 完成
0.16	2026-07-11	発展編 (\$16: LAN 上の別 PC・Hyper-V ホストからの接続) を追記。SSO 構築は不変で、① H TTPS サーバ証明書信頼 (root CA 配布 or 証明書警告の許容)、② ネットワーク通信許可 (実 IP への hosts・仮想スイッチ・ファイアウォール 443/8443) の2観点を整理	ネットワーク到達性の追加のみ
0.17	2026-07-11	\$16.6 「本番でのサーバ証明書	顧客向け証明書ア

版	日付	変更概要	備考
		の用意」を追記（社内 CA/AD C S=パターン A を最有力、パブ リック CA=B、自己署名+配布 =C、および実業務向けアドバ イス6点）。学習用自己署名 C A は本番に持ち込まない旨を明 記	ドバイス
0.18	2026-07-11	発展編\$17「組織内の複数 PLM 環境の SSO 対応（並行運用の設 計メモ）」を追記（HTTP=従来 認証を維持し HTTPS=SSO 用 ポートを新設、1 IdP 共有・1 SP で SSO ポートのみ複数 Site 保護 =方式1・同一ホスト名で証明 書1枚共有・Web.config で認証 方式切替）。\$16.6 挿入時に欠 落していた付録 A 見出しを復 旧	複数環境の並行運 用（設計メモ）
0.19	2026-07-11	\$17 にホスト名設計を反映：構 成図を実ホスト名で具体化 し、17.3(f) を追加（ SP は既存 p lmdev.plm-lab.local に統一し ポートで識別/ IdP は idp.plm-l ab.local に分離/1台に両ホス ト名割当・証明書は役割ごと1 枚）	ホスト名設計の指 針
0.20	2026-07-11	\$6.2 に hosts の実 IP 選択肢を補 足（127.0.0.1 でも実 IP でも可。 Shibboleth はホスト名で判定す るため設定・証明書は不変。実 IP 統一の利点と注意=固定 I P・ファイアウォール）。\$10. 3・\$16 に実機確認結果を反映 （Hyper-V ホストからの SSO 成 功。8080 の localhost 限定は SSO 経路（443→8443）に含まれな いため外部 SSO に影響しな い）	実機確認の反映
0.21	2026-07-12	チェックポイントからの再構 築検証（手順書のみで完走）で 判明した 12 点をまとめて反 映：\$3.1/\$9.2（install.bat の文 は 4 項目のみ。Keystore/Sealer は	再構築検証の反映

版	日付	変更概要	備考
		自動生成 Host Name 既定値 sp. による 注意 /\$6.2 実IP採用時の段階 でファイアウォール 443・844 3 を開放。Test-NetConnection は サービス未起動のため失敗して当然 status が Access Denied による副作用を予告 /\$7. 3 Directory Studio のzip 解凍 Po werShell 手順) /\$7.6 New Sear ch による検索手順を詳細化 /\$9.1 【IdP 展開フォルダは】ネーム不要・理 由 /\$9.4 ⚠️ バックアップを con f/credentials 配下に .propertie s のまま置かない ⇒dap - Copy.p roperties が読み既定値競合し Pool is empty で認証失敗 .orig にする IdP 外 へ trustCertificates/trustStore の コメントアウトを必須に格上げ /\$9.6 ☐ロ グ確認の PowerShell コマンド とメッセージ判断基準の表。Du plicate properties WARN は要確 認、status.vm ERROR は無害 /\$10. 4 ☐確認はメタデータで行う。実 P 運用では status が Access Denied = 想定内 /\$12.3 ☐「サイト全体」= <Site> に登録したサイトの全 パス。登録なしポートはSP管轄外従確認 証のまま /\$13.7 登録 ☐ Pool is em pty and connection creation fai led の切分け手順 /登録-1 (バックアッ プはIdPの外)	
0.22	2026-07-12	/\$9.2 に補足：IdP 5 でキースト ア/Sealer のパスワードが自動生 成される理由（人間用ではな く IdP が自分の鍵を開く内部資 格情報。IdP 3/4 は対話入力 だった）、既定値ではなくイ ンストールごとのランダム値 なので変更不要、文字列だけ 書き換えると鍵と不整合で起 動不能、PFX の changeit とは 別物（自分で作った (a) のファイル vs IdP が作った (b) の鍵）を明	パスワード設計 の理解

版	日付	変更概要	備考
0.23	2026-07-12	記 \$15.8「テスト実施時の留意点と確認手段」を新設（旧 15.8 は 15.9 へ）：(1) 証明書警告の意味（失われるのはサーバの真正性、暗号化は有効。社内テストでの配布省略は許容可だが S P/IdP の 2 回警告・本番は正規証明書）、(2) プライベートウィンドウ = 状態リセットの道具（使わないと「前のセッションで入れただけ」を SSO 成功と誤判定）、(3) 複数環境間の認証スキップの 2 メカニズム（SP Cookie のポート跨ぎ共有 / IdP セッション Cookie = SSO の本質。引き渡し役はブラウザの Cookie でサーバ間通信ではない）、(4) Cookie の中身は暗号化で読めない（引換券。実体はサーバ側）→ 確認は開発者ツール / /Shibboleth.sso/Session/SAML-tracer の 3 手段	テスト実施の理解
0.24	2026-07-12	\$14.8に「アプリから見た認証情報の受け取り方（3 層の役割分担）」を追記：アプリは SAML アサーションも Cookie も見ず、SP が確定した REMOTE_USER を読むだけ（層 1 アサーション = ログイン時 1 回・SP が検証 / 層 2 サーバ側セッション + 暗号化 Cookie = 引換券・アプリは読めない / 層 3 REMOTE_USER = アプリが見る唯一の入口）。ASP / ASP.NET の実装イメージ、追加属性はサーバ変数で渡せる旨も記載	アプリ改修者向けの理解
0.25	2026-07-13	\$17 の認証切替を実態に即して修正（同一フォルダでは Web.config を分けられない → appSettings の AuthMode を IIS のサイト単位設定で上書きし共通	開発環境の構成方針

版	日付	変更概要	備考
		認証関数で分岐。<location>は不可)。\$17.6「開発・ビルド環境の構成」を新設：プロジェクトは1つのまま・同じ物理フォルダを2サイト（80/443）から提供し MainWebSso.aspxのみ追加・子画面は共有かつ無改修（認証処理が共通化済のため改修は共通関数1箇所）／別プロジェクト案が成立しない理由☒SP保護はサイト単位で子画面が保護外・ASP.NET セッション分断）／デバッグ方法（プロセスにアタッチ、#If DEBUGでREMOTE_USERをモック）／各開発者のローカルIISにSPが必要（ホスト名・証明書・メタデータ登録が各自分）／IdPは社内1か所を共有☒1 IdP：多 SP)	
0.26	2026-07-16	実サーバ（Windows Server 2016）での構築知見を\$18として新設：18.1 TLS 1.2の有効化（SC HANNELレジストリ+再起動。IISはOSのTLSを使うため。Tomcat/8443はJava独自で無関係＝切り分けに使える）／18.2 Tomcat サービス導入後のシャットダウン不完了（停止タイムアウトを20秒に設定して解消・切り分け手順）／18.3 OpenSSL 3.xのPKCS#12は-legacyが必要☒Windowsインポートで「パスワード不一致」に化ける・作成/確認の両方で付与）／18.4 証明書はサーバ自身で作るのが確実（Git for Windows）／18.5 既存 Default Web Siteがある環境ではSSO用サイトのIDが1以外になり、shibboleth2.xmlの<Site id>をそのIDに合わせる（今回はid=2）／18.6 ホスト名バインド時のlocalhos	実サーバ構築の知見

版	日付	変更概要	備考
		t 404 は正常・テストは登録ホスト名で・Shibboleth.sso/Stat us は <Site> 登録後に有効	
本書は、WSL 版構築手順書（WSL2 上に IdP スタックを構築した学習環境）を土台に、WSL を一切使わない純 Windows 構成で顧客 PLM 検証環境を再現するための手順書です。SP 側（IIS + Shibboleth SP）と SAML 設計の考え方は WSL 版から流用し、IdP 側（Tomcat/Shibboleth IdP）・LDAP を Windows ネイティブに置き換えています。			

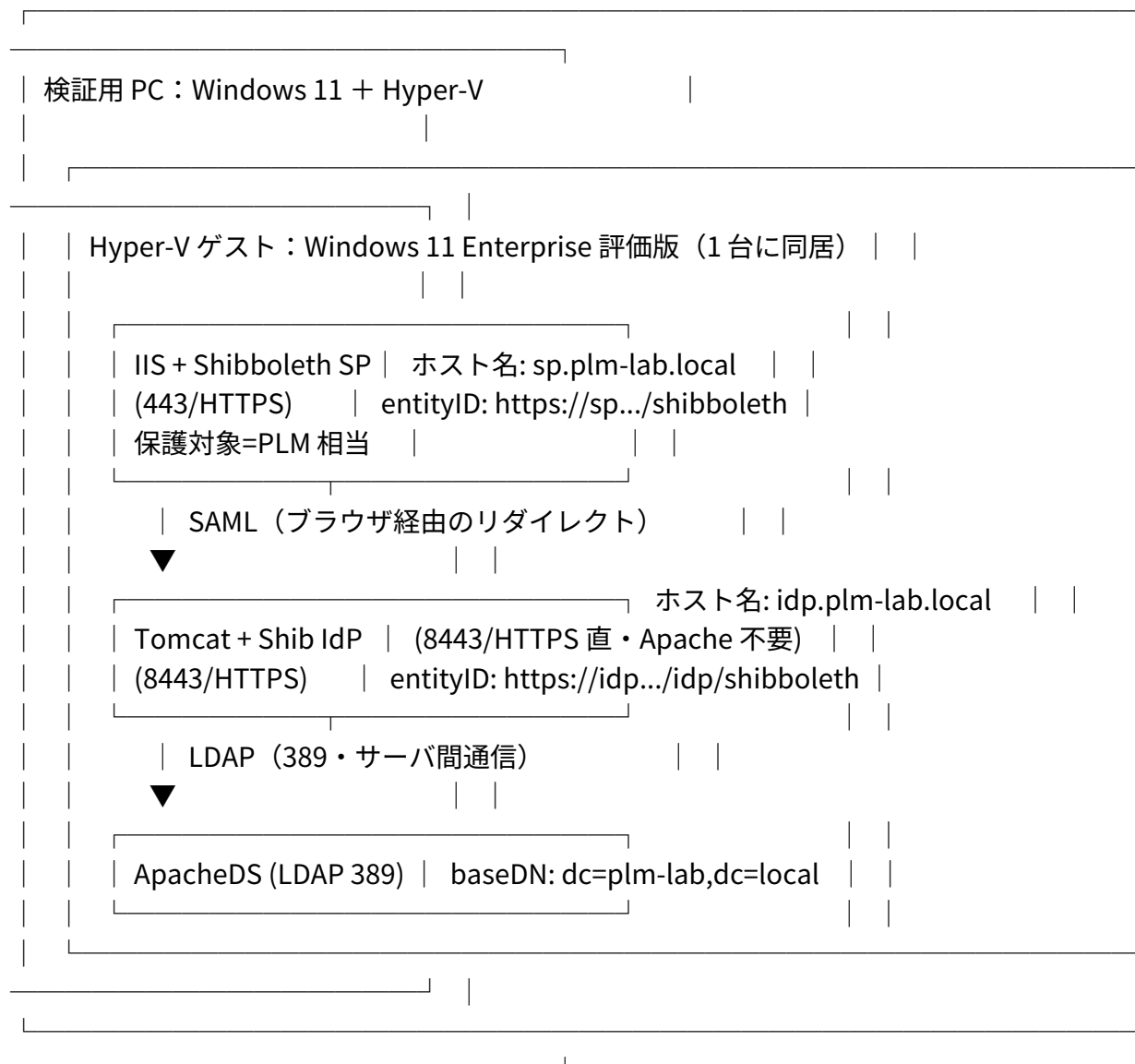
1. 目的と前提

- 目的：顧客 PLM 環境（Shibboleth=SP、前段=IIS、PLM は C:\inetpub\wwwroot 配下の IIS アプリ、IdP=Entra ID／独自認証）に近い SAML SSO 検証環境を、全メンバーが習熟している **Windows** のみで構築し、SP 側の設定と動作を確認する。
- 方針：コンテナや WSL を使わず、すべて **Windows** 上で手作業構築する。IdP は学習用に自前構築し、将来 Entra ID に差し替え可能な形にする。
- 物理ホスト：検証用 PC（Windows 11、Hyper-V 有効）。
- 検証用 **Windows**：Hyper-V 仮想マシン上の **Windows 11 Enterprise** 評価版（90 日 / slmgr/rearm で延長。付録 A）。
- 役割分担（すべて同一のゲスト **Windows** 上に同居）：
 - IIS + Shibboleth SP**：ブラウザからの入口（前段）。未認証を IdP へ、認証後は REMOTE_USER を確定し、wwwroot 配下の PLM 相当アプリを実行。
 - Tomcat + Shibboleth IdP**：学習用テスト IdP（HTTPS 8443 を直接公開）。将来 Entra ID に差し替え可能。
 - ApacheDS (LDAP)**：ユーザーディレクトリ。IdP が認証・属性取得のために参照。
- 前提（電源管理）：物理ホスト・ゲスト仮想マシンとも **Windows** のスリープ／休止は無効化していること（組織の対象 PC も同様の運用）。本書はこの前提に立ち、スリープ復帰に伴う時刻ずれ対策は主手順から除外する（参考手順は付録 C）。
- 顧客環境との対応（確認済み事項）：
 - 前段 = **IIS**、SP = **Shibboleth SP**（**IIS** ネイティブモジュール）、PLM = **IIS** 上のアプリ（顧客と一致）。
 - IdP = **Entra ID**（+ 独自認証）。識別子は **emailAddress** 形式（例 01PLM01@plm-lab.local）。
 - PLM は将来、メール形式の識別子を受け取り @ の前を切り出して従来の識別番号として扱う想定（**PLM** アプリ側の責務であり、本書の SP/IdP 構築範囲外）。

⚠ 重要な前提：本環境は Windows 仮想マシン 1 台にすべてが同居します。仮想マシンを削除すると IdP/LDAP/SP すべてが失われます。評価版の期限は

削除・再インストールではなく slmgr/rearm で延長してください（付録 A）。
作業成果の保全・再構築検証の考え方は付録 B を参照。

2. 全体アーキテクチャ



SAML Web SSO の流れ (未認証時) :

1. ブラウザが IIS[SP・443] の保護対象 (PLM 相当) にアクセスする。
2. SP が「未認証」と判断し、ブラウザを IdP[8443] へリダイレクトする。
3. ブラウザが IdP[Tomcat・8443] のログイン画面にアクセスする。
4. IdP が ApacheDS[LDAP・389] に問い合わせてユーザーを認証する (サーバ間通信)。
5. IdP が SAML アサーションを、ブラウザ経由で SP の ACS へ POST する。

6. SP がアサーションを検証してセッションを確立し、REMOTE_USER を確定する。
7. IIS が wwwroot 配下の PLM 相当アプリを実行して応答する。

WSL 版との違い：IdP 側が Windows ネイティブのため、IdP 前段の **Apache H TTPD** は不要 (Tomcat が HTTPS 8443 を直接公開)。WSL2 の mirrored ネットワーク・localhost 転送・ポート衝突といった問題は発生しない。IIS (SP) と Tomcat (IdP) は同一 Windows 上に同居するが、互いに直接は通信せず、ブラウザのリダイレクトと事前交換したメタデータで連携する。LDAP を参照するのは IdP のみ。

3. パラメータ一覧 (確定・調整用)

本書全体で参照する値。着手前に内容を確認し、変更する場合はここを起点に各フェーズへ反映してください。

項目	既定値	備考
内部ドメイン	plm-lab.local	検証用の架空ドメイン
IdP ホスト名	idp.plm-lab.local	Tomcat (IdP) 側
SP ホスト名	sp.plm-lab.local	IIS (SP) 側
Java	OpenJDK 17 (Windows 版)	Shibboleth IdP 5 の要件。Temurin 等
Servlet コンテナ	Apache Tomcat 10.1.x (Windows サービス)	IdP 5 は Tomcat 10.1 必須 (9 系不可)
IdP	Shibboleth IdP 5.x	Java 17 / Jakarta 名前空間
SP	Shibboleth SP 3.x (IIS ネイティブモジュール)	IIS 用
LDAP	ApacheDS (Java 製・Windows サービス)	Apache Directory Studio で投入。inetOrgPerson
IdP entityID	https://idp.plm-lab.local/idp/shibboleth	フェーズ 5 で確定
SP entityID	https://sp.plm-lab.local/shibboleth	フェーズ 8 で確定
LDAP ベース DN	dc=plm-lab,dc=local	ApacheDS (フェーズ 3)
LDAP 検索バインド	uid=idp-reader,ou=people,dc=plm-lab,dc=local	IdP の LDAP 検索用 (読取専用)
テストユーザー	uid=01PLM01 / uid=01PLM02	ou=people 配下。識別番号 (社内テスト環境の形式に準拠)

項目	既定値	備考
テストユーザーの mail	01PLM01@plm-lab.local / 01PLM02@plm-lab.local	emailAddress 形式 NameID の元 (顧客 Entra の形式に準拠)
NameID 形式	emailAddress	顧客 (Entra ID) の形式に合わせる (WSL 版の unspecified から変更)
REMOTE_USER	優先リスト (例 azure_id … gid_id に倣う)	メール形式 NameID を載せる。学習では 1~2 個で可
IdP ホーム	C:\opt\shibboleth-idp	Shibboleth IdP 5 (Windows・フェーズ 5)
Tomcat ホーム	C:\opt\tomcat (例)	Windows サービスとして稼働 (フェーズ 4)
SP インストール先	C:\opt\shibboleth-sp	既定。C:\inetpub 配下は不可 (鍵・設定の漏洩リスク)
PLM アプリ配置	C:\inetpub\wwwroot 配下	検証では REMOTE_USER 表示のテストページ
SP ポート (HTTPS)	443	IIS (SP)
IdP ポート (HTTPS)	8443	Tomcat (IdP) 直公開。同一 Windows 上での 443 衝突回避
内部 CA ルート名	PLM-Lab Root CA	フェーズ 2 で作成。idp/sp のサーバ証明書を発行
証明書作成ツール	openssl for Windows (Git for Windows 同梱等)	WSL 版の openssl 手順を流用
ゲスト VM メモリ	8 GB (目安)	フルスタック同居のため
ゲスト VM vCPU	4	目安
タイムゾーン	Asia/Tokyo	ゲスト Windows

ホスト名は当面 hosts ファイルで解決します (DNS サーバは立てません)。SP・IdP とともに同一ゲスト Windows 上のため、127.0.0.1 で解決させます (フェーズ 2)。

2 種類の証明書 (再掲・重要) : (a) TLS/HTTPS サーバ証明書 (ブラウザに信頼される・SAN 一致必須。内部 CA で idp/sp を発行) と、(b) SAML 署名・暗号化証明書 (メタデータ交換で信頼・自己署名可・CA 信頼もホスト名一致も不要。IdP はインストール時に自動生成、SP は keygen で生成)。準備が要るのは (a)。詳細はフェーズ 2 で扱う。

3.1 アカウント・パスワード一覧（わかりやすさ優先）

本検証環境ではわかりやすさを優先し、既定値や「Joe アカウント（uid とパスワードが同じ）」を採用する。いずれも検証専用であり、実際の環境構築時にはセキュリティを考慮して強固な値に変更すること。OS ユーザーとしての専用アカウント（WSL 版の tomcat 相当）は Windows では作らず、各サービスは Windows のサービスアカウント（Local System 等）で動作する。構築作業はローカル管理者（本書では **Administrator**。読み替え可）で、昇格した PowerShell / コマンドプロンプトで行う。

#	アカウント／鍵	値（検証用）	使用箇所	フェーズ
1	ApacheDS 管理者	uid=admin,ou=system / secret	ApacheDS 管理（既定値のまま）	3
2	テストユーザー 1	uid=01PLM01 / パスワード 01PLM01	ログイン検証 ☒Joe アカウント）	3
3	テストユーザー 2	uid=01PLM02 / パスワード 01PLM02	2 人目・再現性検証（Joe アカウント）	3
4	IdP 検索バインド	uid=idp-reader / パスワード idp-reader	IdP の LDAP 検索（読取専用）	3・5
5	IdP キーストア	自動生成（対話で聞かれない。credentials\secrets.properties に格納）	IdP install.bat	5
6	IdP Sealer	自動生成（同上）	IdP install.bat	5
7	TLS 証明書 PFX☒idp/sp)	changeit	idp.pfx / sp.pfx の作成・取込 ☒Tomcat・IIS)	2・6・7

実機で使用したインストーラ／パッケージ（検証時点の版。最新版があれば読み替え可）：OpenJDK17U-jdk_x64_windows_hotspot_17.0.19_10.zip☒Temurin 17）、apacheds-2.0.0.AM27.exe☒ApacheDS）、ApacheDirectoryStudio-2.0.0.v20210717-M17-win32.win32.x86_64.zip☒Directory Studio）、apache-tomcat-10.1.5-7-windows-x64.zip☒Tomcat）、shibboleth-identity-provider-5.2.3.zip☒IdP）、shibboleth-sp-3.5.2.3-win64.msi☒SP）。

OS ユーザーの扱い：構築は Administrator（ローカル管理者）。Tomcat/IdP・ApacheDS・shibd☒Shibboleth Daemon）はいずれも Windows サービスとして Local System 等のサービスアカウントで動作するため、Linux の tomcat ユーザーのような専用 OS ユーザーの作成・所有権付与は不要。

4. 構築フェーズ全体像（ロードマップ）

フェーズ	内容	WSL 版との関係	本書での状態
1	検証環境の初期設定（ゲスト Windows VM・時刻同期・前提）	新規（WSL 導入を廃し簡素化）	✓ 本版で記載
2	ネットワーク土台（hosts・内部 CA・idp/sp 証明書（a）・ポート設計）	変更（openssl for Windows で流用）	✓ 本版で記載
3	ApacheDS (LDAP) 導入・ディレクトリ設計・テストユーザ投入	新規（OpenLDAP から置換）	✓ 本版で記載
4	OpenJDK + Tomcat (Windows サービス)	変更（Windows 版・service.bat）	✓ 本版で記載
5	Shibboleth IdP 5 (Windows・LDAP 連携・emailAddress NameID 準備)	変更（install.bat・Windows パス）	✓ 本版で記載
6	Tomcat 直 HTTPS (8443) 公開	置換（Apache 前段を廃止）	✓ 本版で記載
7	IIS (SP) の	流用（WSL 版とほぼ同一）	✓ 本版で記載

フェーズ	内容	WSL 版との関係	本書での状態
8	保護対象 サイト・ 443/TLS) Shibboleth SP⇔IIS ネ イティブ モジュー ル・サイ ト全体保 護)	流用 (WSL 版とほぼ同一)	✓ 本版で記載
9	メタデー タ交換 (I dP ⇄ SP の相互信 頼・初回 SSO)	変更 (:8443 補正が不要に)	✓ 本版で記載
10	属性連携 ⇔emailAdd ress 形式 NameID を REMOT E_USER に 載せる)	変更 (unspecified⇄emailAddress)	✓ 本版で記載
11	結合テス ト (ログ イン・再 現性・再 起動堅牢 性・ロ グ)	流用／変更	✓ 本版で記載

各フェーズは「目的 → 前提 → 手順 → 動作確認」の順で記載します。付録：A (評価版 rearm) / B (バックアップ・再現性検証のチェックポイント運用) / C (スリープ環境向け時刻再同期) / D (トラブルシュート・Windows 固有) / E (オフライン導入)。

5. フェーズ 1：検証環境の初期設定

目的：Hyper-V ゲストの Windows 11 を、以降の全スタック (ApacheDS / Tomcat + IdP / IIS + SP) を同居させる土台として整える。あわせて、SAML でつまずきやすい時刻同期を最初に固め、やり直しに備えた着手前チェックポイントを取得する。

WSL 版との違い：本構成は WSL2 を使わないため、入れ子（ネステッド）仮想化の有効化は不要。ゲスト Windows 上で Java/Tomcat/IIS が動くだけで、通常の Hyper-V ゲストとして扱える（WSL 版 \$5.2 の ExposeVirtualizationExtensions や MAC スプーフィングは本版では不要）。

5.1 事前確認

- ・ 検証用 PC で Hyper-V が有効で、ゲストの Windows 11 評価版が作成済みであること。
- ・ ゲストに割り当てるメモリ（8GB 目安）・vCPU^④4 目安）の余裕が検証用 PC にあること。
- ・ ゲスト Windows にローカル管理者（**Administrator**。自宅では読み替え）でサインインできること。
- ・ 物理ホスト・ゲストとも **Windows** のスリープ／休止が無効であること（\$1 の前提）。

5.2 ゲスト VM の電源オプションと着手前チェックポイント

やり直しに備え、素の状態のチェックポイントを取得します。まず、電源断時に保存状態へ入らずシャットダウンさせる設定にしてから、VM を停止した状態でチェックポイントを取得します。検証用 PC（物理ホスト）の管理者権限 **PowerShell** で実行します。

#1) 対象 VM 名を確認

Get-VM

#2) 電源断時に保存状態にせずシャットダウンさせる（保存状態からの復帰による時刻ずれ回避）

Set-VM -Name "<VM 名>" -AutomaticStopAction ShutDown

#3) 種類を標準（Standard）チェックポイントに（本構成は入れ子仮想化を使わないため Standard で可）

Set-VM -Name "<VM 名>" -CheckpointType Standard

#4) ゲストをシャットダウン（停止状態で取得するのが最も安全）

Stop-VM -Name "<VM 名>"

#5) 着手前の素の状態を取得

Checkpoint-VM -Name "<VM 名>" -SnapshotName "Phase1 前_素の Windows11"

 **パラメータ名の注意（-Name と -VMName）**：VM 自体を操作する系（Set-VM / Get-VM / Start-VM / Stop-VM / Checkpoint-VM）は VM 名を **-Name** で指定する。VM の構成要素を操作する系（Set-VMProcessor / Set-VMMemory 等）は **-VMName** だが、本フェーズでは使用しない。取り違えを避けたい場合は Get-VM "<VM 名>" | Set-VM -CheckpointType Standard のようにパイプで渡す。

チェックポイントは短期のやり直し用でありバックアップではありません（付録 B）。本構成は入れ子仮想化を使わないため、標準（Standard）チェックポイントが利用でき、稼働中の取得も比較的安全ですが、着手前の素の状態は停止して取得するのが確実です。

5.3 時刻同期の確認

SAML は IdP と SP の時刻の一致に敏感で、ズレると「clock skew」エラーの原因になります。本構成では IdP（Tomcat）も SP（IIS）も同一のゲスト **Windows** 上にあるため、両者の時刻は常に一致し、WSL 版のような OS 間の相対ずれは原理的に発生しません。したがって、ゲスト Windows の時計が正しく保たれていれば十分です。

- **Hyper-V** 統合サービス「時刻同期」が有効であることを確認（既定で有効）。これによりゲスト Windows は検証用 PC（ホスト）の時計に追従します。ゲストの管理者 PowerShell で確認：

タイムゾーンが東京か

Get-TimeZone

東京でなければ設定

Set-TimeZone -Name "Tokyo Standard Time"

Windows Time サービスが稼働しているか（オンライン時は NTP 同期も有効）

w32tm /query /status

オンライン環境なら Windows Time サービス（w32tm）が NTP に同期します。オフライン環境でも、IdP と SP が同一 OS 上のため相対ずれは発生せず、SAML の clock skew は問題になりません。スリープを使う環境で運用する場合の復帰時再同期は付録 C を参照。

5.4 導入方針と作業用フォルダの準備

導入方針（本書全体）：**zip** パッケージが選べるものは **zip** を展開して導入し、環境変数など必要な設定は手作業で行う（どこに何が入ったかが明確になり、再現・撤去・切り分けが容易）。導入物は原則 **C:\opt** 配下・空白を含まないパスに統一する（Java アプリでの空白起因トラブルを避ける）。作業は昇格した PowerShell / コマンドプロンプトで行う。

例外：ApacheDS 本体は zip 提供が無いためインストーラ（.exe）で導入する（§7.2）。

作業用フォルダを用意します。

New-Item -ItemType Directory -Force -Path C:\opt, C:\lab, C:\lab\ca, C:\lab\installers | Out-Null

- C:\opt : jdk-17 / directory-studio / tomcat / shibboleth-idp / shibboleth-sp の導入先。
- C:\lab\ca : 内部 CA ・証明書の作成場所（フェーズ 2）。

- C:\lab\installers : 入手した各インストーラ/zip の保管。

5.5 動作確認チェックリスト (フェーズ 1)

#	確認内容	コマンド / 方法	期待結果
1	ゲストにローカル管理者でサインイン	—	Administrator (読み替え可) でサインイン済み
2	タイムゾーン	Get-TimeZone	Tokyo Standard Time
3	時刻サービス	w32tm /query /status	稼働 (オンライン時は NTP 同期)
4	着手前チェックポイント	Hyper-V マネージャー	Phase1 前_素の Windows11 が存在
5	昇格実行の確認	管理者 PowerShell が使える	昇格プロセスでコマンド実行可

5.6 OpenJDK (Temurin 17) の導入 (ApacheDS・Tomcat の前提)

ApacheDS (フェーズ 3) と Tomcat (フェーズ 4) はいずれも Java で動作する。**Java** は両者の共通基盤のため、ここ (フェーズ 3 の前) で先に導入しておく (ApacheDS インストーラが JAVA_HOME を求めるため、順序として Java が先)。

1. 入手 : Adoptium (https://adoptium.net/) から **Temurin 17 (LTS) ・ Windows x64 ・ JDK ・ zip** を入手し C:\lab\installers へ。
2. 展開 & リネーム (管理者 PowerShell) :

```
Expand-Archive -Path "C:\lab\installers\OpenJDK17U-jdk_x64_windows_hotspot_*.zip" -DestinationPath "C:\opt" -Force
Get-ChildItem C:\opt | Where-Object Name -like "jdk-17*"      # 展開フォルダ名を確認
Rename-Item "C:\opt\jdk-17.0.xx+xx" "C:\opt\jdk-17"         # 分かりやすく (任意)
```

3. 環境変数 (システム) を手動設定 :

```
[Environment]::SetEnvironmentVariable("JAVA_HOME", "C:\opt\jdk-17", "Machine")
$P = [Environment]::GetEnvironmentVariable("Path", "Machine")
[Environment]::SetEnvironmentVariable("Path", "$P;C:\opt\jdk-17\bin", "Machine")
```

設定後は新しい **PowerShell** を開き直す (既存セッションには反映されない)。

4. 確認 : java -version (openjdk version "17...")、echo \$env:JAVA_HOME。

この後にフェーズ 2 (証明書) ・ フェーズ 3 (ApacheDS) へ進む。ApacheDS インストーラが JAVA_HOME を尋ねたら C:\opt\jdk-17 を指定する。

すべて確認できれば、フェーズ 1 は完了です。次はフェーズ 2 (ネットワーク土台 ・ 内部 CA と idp/sp 証明書の発行) です。

6. フェーズ 2：ネットワーク土台 (hosts・内部 CA・idp/sp サーバ証明書)

目的：コンポーネント導入の前に、全体が依存する「名前解決 (hosts)」と「TLS 証明書」を先に確定する。純 Windows 構成では SP(IIS) も IdP(Tomcat) も同一のゲスト Windows 上にあり、いずれも 127.0.0.1 で解決するため、WSL 版のようなネットワーク方式の選定 (NAT/mirrored の段階移行) や、localhost 転送・ポート転送に関する検討は不要になる。

WSL 版との違い：WSL 版では「ブラウザー-WSL2 の到達性」を確保するため mirrored モードや (A)→(B) 段階移行を検討したが、本構成では IdP が Windows ネイティブのため、その検討は丸ごと不要。ポート設計 (SP=443/IdP=8443) だけは同じ考え方で踏襲する (同一ホスト上で 443 を取り合わないため)。

6.1 ポート設計

同一のゲスト Windows 上で SP(IIS) と IdP(Tomcat) が 443 を取り合わないよう、IdP を別ポート (8443) にします。

役割	ホスト名	URL (ベース)	ポート	バインド先
SP	sp.plm-lab.local	https://sp.plm-lab.local/	443	IIS
IdP	idp.plm-lab.local	https://idp.plm-lab.local:8443/idp/	8443	Tomcat (直 HTTP S)

SAML のエンドポイントはすべて URL なので、ポートが異なっても問題ありません。顧客の本番 (IdP=Entra ID) では IdP は Entra 側の URL になるため、この 8443 は「学習用テスト IdP を同一ホストに同居させるための便宜」です。

6.2 名前解決 (hosts)

DNS サーバは立てず、hosts で解決します。SP・IdP とともに同一のゲスト Windows 上のため、両方を 127.0.0.1 に向けます。

ゲスト Windows : C:\Windows\System32\drivers\etc\hosts 管理者権限で編集。昇格したメモ帳等)

```
127.0.0.1 sp.plm-lab.local
127.0.0.1 idp.plm-lab.local
```

管理者 PowerShell で追記する場合：

```
Add-Content -Path "$env:SystemRoot\System32\drivers\etc\hosts" -Value "127.0.0.1`tsplm-lab.local`r`n127.0.0.1`tidp.plm-lab.local"
```

127.0.0.1 と実 IP のどちらでもよい：hosts は「ホスト名-IP」の対応表にすぎず、重要なのはその IP で **IIS (443) / Tomcat (8443)** に到達できるかだけ。ゲスト自身の NIC に割り当てた固定 IP (例 **192.168.x.x**) を書いても **SSO** は同じように動作する (Shibboleth は IP ではなくホスト名で判定するため、SP の <Site name=...>・entityID・メタデータ・証明書 SAN はいずれも影響を受けない。証明書の作り直しも不要)。

実 IP に統一する利点：発展編 (§16) で Hyper-V ホストや LAN の別 PC から接続する場合、すべてのマシンの **hosts** を同じ実 IP で統一でき、記述が揃う (127.0.0.1 のままだとゲスト内は自分・外部 PC は実 IP と分かれて混乱しやすい)。本番 (DNS で実 IP に解決) の構造にも近い。

実 IP にする場合の注意：① IP が固定であること (DHCP で変わると hosts と不整合)、②ファイアウォールの受信許可が必要になる (ループバック宛は素通りするが、実 IP 宛は受信規則の対象)。

実 IP を採用する場合は、この段階でファイアウォールも開けておく (後続フェーズで「なぜか繋がらない」を防ぐ)。サービス (IIS 443 / Tomcat 8443) が未起動でも、受信規則は先に作ってよい (規則は許可の宣言にすぎず、待ち受けが無くても無害)。

受信許可 (443=SP/IIS、8443=IdP/Tomcat)

```
New-NetFirewallRule -DisplayName "Allow HTTPS 443 (SP)" -Direction Inbound -Protocol TCP -LocalPort 443 -Action Allow
```

```
New-NetFirewallRule -DisplayName "Allow HTTPS 8443 (IdP)" -Direction Inbound -Protocol TCP -LocalPort 8443 -Action Allow
```

規則が登録されたことを確認

```
Get-NetFirewallRule -DisplayName "Allow HTTPS*" | Select-Object DisplayName, Enabled, Direction, Action
```

この時点で **Test-NetConnection ... -Port 443/8443** は失敗して当然 (443=IIS はフェーズ 7、8443=Tomcat はフェーズ 6 で導入するため、まだ誰も待ち受けていない)。本フェーズで確認するのは「名前解決 (ping で実 IP に解決される)」と「規則が登録されたこと」まで。疎通確認 (**Test-NetConnection**) は各サービス構築後に行う (8443-§10.4、443-§11.6)。

実 IP を採用した場合の副作用 (予告)：IdP の管理系エンドポイント /idp/profile/status は、既定で **localhost (127.0.0.1:::1)** からのみ許可されている (conf/access-control.xml)。hosts を実 IP にすると、実 IP 宛のアクセスとなり **Access Denied** が返る (TLS・接続・証明書は正常。SSO 本体には影響しない)。status の確認は <http://localhost:8080/idp/profile/status> で行い、8443 の HTTPS 確認はアクセス制御の影響を受けないメタデータ <https://idp.plm-lab.local:8443/idp/shibboleth> で行う (§10.4 参照)。

確認：

```
ping sp.plm-lab.local # 127.0.0.1 (または設定した実 IP) に解決されること
ping idp.plm-lab.local # 127.0.0.1 に解決されること
```

6.3 【参考】この構成で登場する 2 種類の証明書

Shibboleth では性格の異なる 2 種類の証明書が登場し、混同しやすいポイントです。役割を分けて理解してください。

観点	(a) TLS/HTTPS サーバ証明書	(b) SAML 署名・暗号化証明書
目的	ブラウザーサーバの HTTPS 通信の暗号化とサーバ真正性	SAML メッセージ (アサーション等) の署名・暗号化
使う場所	IIS [Ⓐ] SP, 443) 、Tomcat [Ⓐ] IdP, 8443)	IdP・SP の SAML 処理内部
誰が検証するか	ブラウザ	相手方の IdP/SP
信頼の確立方法	CA を信頼ストアに登録 (PKI)	メタデータ交換で公開鍵を相互登録
ホスト名 ⒶSA N) 一致	必須	不要
CA 署名	必要 (本書では内部 CA で発行)	不要 (自己署名が通常)
準備するタイミング	フェーズ 2 (本節)	各コンポーネント導入時に生成 (IdP=フェーズ 5、SP=フェーズ 8)

本節 (6.4) で用意するのは (a) のみ。(b) は後続フェーズでコンポーネントが生成します。両者は独立で、(b) に CA 信頼やホスト名一致は不要です。

6.4 内部 CA の作成と idp/sp サーバ証明書の発行 ((a) の準備)

Git for Windows 同梱の **openssl** を使います。**Git Bash** を起動して以下を実行します[Ⓐ]WSL 版と同じ openssl 手順をそのまま流用)。作業場所は \$5.4 で作成した C:\lab\ca[Ⓐ]Git Bash では /c/lab/ca) 。

```
cd /c/lab/ca
```

1) 内部 CA ルート (秘密鍵と自己署名証明書、10 年)

```
openssl genrsa -out rootCA.key 4096
openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 \
-subj "//C=JP\O=PLM-Lab\CN=PLM-Lab Root CA" -out rootCA.crt
```

2) idp のサーバ証明書

```
cat > idp.ext <<'EOF'
subjectAltName = DNS:idp.plm-lab.local
extendedKeyUsage = serverAuth
EOF
openssl genrsa -out idp.key 2048
openssl req -new -key idp.key -subj "//C=JP\O=PLM-Lab\CN=idp.plm-lab.local" -out idp.csr
openssl x509 -req -in idp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
-out idp.crt -days 825 -sha256 -extfile idp.ext
```

3) sp のサーバ証明書

```
cat > sp.ext <<'EOF'
subjectAltName = DNS:sp.plm-lab.local
extendedKeyUsage = serverAuth
EOF
openssl genrsa -out sp.key 2048
openssl req -new -key sp.key -subj "//C=JP\O=PLM-Lab\CN=sp.plm-lab.local" -out sp.csr
openssl x509 -req -in sp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
-out sp.crt -days 825 -sha256 -extfile sp.ext
```

⚠ **Git Bash** 特有の注意 (**MSYS** のパス変換) : Git Bash では -subj "/C=JP/..." の先頭の / が Windows パスに誤変換されることがあります。上記のように先頭を // にし、区切りを \\\\C=JP\\O=PLM-Lab\\CN=..." とすると回避できます。うまくいかない場合は、環境変数 MSYS_NO_PATHCONV=1 を付けて実行 (例 : MSYS_NO_PATHCONV=1 openssl req ...) してもよいです。

PFX (PKCS12) 化 : IIS(sp) と Tomcat(idp) は、HTTPS バインドに証明書 + 秘密鍵を **PFX** 形式で取り込みます。両方を作成します (エクスポートパスワードは \$3.1 のとおり changeit) 。

IIS (フェーズ 7) 用

```
openssl pkcs12 -export -out sp.pfx -inkey sp.key -in sp.crt -certfile rootCA.crt -passout pass:
changeit
```

Tomcat/IdP (フェーズ 6) 用

```
openssl pkcs12 -export -out idp.pfx -inkey idp.key -in idp.crt -certfile rootCA.crt -passout pa
ss:changeit
```

6.4.1 【解説】証明書コマンドの意味

\$6.4 の各コマンドが「何をしているか」を、後から理解・説明できるよう解説します。全体像は「自前の認証局 (CA) を 1 つ作り、その CA で **idp** と **sp** のサーバ証明書に署名する」という PKI の縮小版です。

3 ステップの関係

[1] 内部 CA ルートを作る (rootCA.key / rootCA.crt) ... 署名する側 (親)

└─[2] idp のサーバ証明書を CA に署名してもらう (idp.key / idp.crt)

└─[3] sp のサーバ証明書を CA に署名してもらう (sp.key / sp.crt)

サーバ証明書 (idp.crt/sp.crt) は単体では信頼されず、信頼された **CA (rootCA)** の署名が付いて初めてブラウザに信頼される。だから先に CA を作り、その CA で各サーバ証明書に署名する。

[1] 内部 CA ルート

- openssl genrsa -out rootCA.key 4096 : CA の秘密鍵を生成。4096 は鍵長 (CA は大元なのでサーバ証明書の 2048 より長く強度を確保)。生成物 rootCA.key は最重要機密 (これで任意の証明書に署名できる)。
- openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 -subj "...CN=PLM-Lab Root CA" -out rootCA.crt : 秘密鍵を使って CA 自身のルート証明書を作る (自己署名)。
 - -x509 : CSR でなく証明書そのものを出力 (自己署名になる)。-new : 新規。-nodes : 秘密鍵をパスフレーズで暗号化しない (起動時のパスワード入力を不要にする)。-key : 署名に使う秘密鍵。-sha256 : 署名ハッシュ。-days 3650 : 有効期間約 10 年 (CA は長寿命)。-subj : サブジェクト (C 国 / O 組織 / CN = この CA の名前。Windows の信頼ストアに表示される名前になる)。先頭 // と区切り \ は Git Bash のパス誤変換回避。
 - 生成物 rootCA.crt : CA の公開ルート証明書 (公開可)。各マシンの信頼ストアに登録して「この CA が署名した証明書を信頼する」状態を作る。

[2] idp のサーバ証明書

- cat > idp.ext ... : 証明書に付ける拡張を書いたファイル。subjectAltName = DNS: idp.plm-lab.local **SAN**。現代のブラウザは CN でなく SAN でホスト名一致を判定するため必須)、extendedKeyUsage = serverAuth **サーバ認証用途**)。
- openssl genrsa -out idp.key 2048 : idp サーバの秘密鍵 (サーバ用は 2048)。
- openssl req -new -key idp.key -subj "...CN=idp.plm-lab.local" -out idp.csr : **CSR** (証明書署名要求 = 申請書) を作成。CSR には idp の公開鍵とサブジェクトが入る (この時点では CA 署名なし)。
- openssl x509 -req -in idp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial -out idp.crt -days 825 -sha256 -extfile idp.ext : **CA** が申請書に署名して正式なサーバ証明書を発行 (中核)。

- x509 -req : 入力が CSR で、署名して証明書を出すモード。-CA/-CAkey : 署名に使う CA 証明書と CA 秘密鍵。-CAcreateserial : シリアル番号管理ファイル (rootCA.srl) を作成/更新。-days 825 : 有効期間 (ブラウザ制限に合わせ短め)。-extfile idp.ext : SAN・用途を証明書に付与 (忘れるとホスト名不一致警告)。
- 生成物 idp.crt : idp の公開鍵 + サブジェクト + SAN + CA の署名が入った正式なサーバ証明書。

[3] sp のサーバ証明書

[2] の idp を sp に置き換えただけで意味は同じ (SAN=DNS:sp.plm-lab.local、CN=sp.plm-lab.local)。同じ **CA** (**rootCA**) で署名するため、rootCA を 1 つ信頼登録すれば idp・sp 両方が信頼される。

生成ファイルまとめ

ファイル	種類	機密性	用途
rootCA.key	CA 秘密鍵	最重要機密	各サーバ証明書への署名
rootCA.crt	CA ルート証明書	公開可	信頼の起点。各マシンの信頼ストアに登録
rootCA.srl	シリアル管理	—	証明書ごとの通し番号 (自動)
idp.key / sp.key	サーバ秘密鍵	機密	Tomcat(IdP)・IIS(SP) の TLS 復号
idp.csr / sp.csr	署名要求	—	発行時の中間ファイル
idp.ext / sp.ext	拡張設定	—	SAN・用途を付与する中間ファイル
idp.crt / sp.crt	サーバ証明書	公開可	Tomcat(IdP,8443)・IIS(SP,443) の HTTPS
idp.pfx / sp.pfx	証明書 + 秘密鍵 (PKCS12)	機密 (鍵を含む)	Tomcat・IIS へのインポート用

要点 : CA を自前で 1 つ作り rootCA を各マシンに登録すれば、内部のサーバ証明書がすべて信頼される。手順は「秘密鍵 - CSR - CA 署名 - 証明書」で、公的証明書取得の縮小版。SAN が無いと鍵マークにならないため subjectAltName が肝。鍵長は CA=4096 / サーバ=2048 と使い分ける。ここで作るのは (a) TLS

/HTTPS サーバ証明書のみで、(b) SAML 署名・暗号化証明書はフェーズ 5・8 でコンポーネントが生成する（別物）。

6.5 rootCA をゲスト Windows の信頼ルートに登録

ブラウザ（Edge/Chrome は Windows 証明書ストアを使用）で鍵マークにするため、内部 CA ルートを「信頼されたルート証明機関」に登録します。管理者 PowerShell で：

```
Import-Certificate -FilePath "C:\lab\ca\rootCA.crt" -CertStoreLocation Cert:\LocalMachine\Root
```

別マシン（検証用 PC = ホストや LAN 上の PC）のブラウザからアクセスする「発展編」を行う場合は、そのマシンの証明書ストアにも同じ rootCA.crt を登録すれば、ホスト名にひも付く証明書はそのまま使えます（作り直し不要）。ただし本書の基本構成はゲスト Windows 上のブラウザで検証します。

6.6 動作確認チェックリスト（フェーズ 2）

#	確認内容	方法	期待結果
1	hosts 解決	ping idp.plm-lab.local / ping sp.plm-lab.local	いずれも 127.0.0.1
2	CA・証明書生成	ls /c/lab/ca (Git Bash)	rootCA.crt / idp.crt / sp.crt / idp.pfx / sp.pfx 等が存在
3	SAN の確認	openssl x509 -in idp.crt -noout -text \ grep -A1 "Subject Alternative"	DNS:idp.plm-lab.local
4	証明書の検証	openssl verify -CAfile rootCA.crt idp.crt / sp.crt	OK
5	Windows の CA 信頼	certlm.msc →「信頼されたルート証明機関」	PLM-Lab Root CA が存在

この時点では証明書を使う Web サーバ（Tomcat/IIS）がまだ無いため、HTTP S 応答の確認は各コンポーネント導入後（フェーズ 6・7）に行います。

7. フェーズ 3 : ApacheDS (LDAP ユーザーディレクトリ)

目的：IdP が認証・属性取得を行うユーザーの元データを、**ApacheDS** (Java 製 LDAP サーバ・Windows で動作) に用意する。ディレクトリ設計 (baseDN・ou=people・テスト)

トユーザー・検索バインド) は WSL 版の OpenLDAP と同じ考え方を踏襲し、製品を ApacheDS に置き換える。ユーザー投入は **Apache Directory Studio** GUI) または **LDIF** インポートで行う。

WSL 版との違い : OpenLDAP apt・slapd・dpkg-reconfigure・LDIF+ldapadd) から ApacheDS Windows インストーラ・GUI ツール) に置き換わる。設計値 (dc=plm-lab,dc=local/ou=people/inetOrgPerson/uid・mail・userPassword) は同じ。メール形式の識別子に対応するため、各ユーザーに **mail** 属性 (01PLM01@plm-lab.local) を持たせる点が新しい。

7.1 ディレクトリ設計

dc=example,dc=com ← ベース DN (ApacheDS 既定パーティションを流用)
|—— ou=people ← ユーザー・検索アカウント
| |—— uid=01PLM01 (inetOrgPerson) ← テストユーザー 1 (mail: 01PLM01@plm-lab.local)
| |—— uid=01PLM02 (inetOrgPerson) ← テストユーザー 2 (mail: 01PLM02@plm-lab.local)
| |—— uid=idp-reader (inetOrgPerson) ← IdP の検索バインド用 (読取)
|—— (将来) ou=groups ← ロール等 (本書では未使用)

項目	値
パーティション suffix(baseDN)	dc=example,dc=comApacheDS 既定パーティションを流用。7.4 参照)
ユーザー OU	ou=people,dc=example,dc=com
テストユーザー	uid=01PLM01 / uid=01PLM02objectClass: inetOrgPerson)。パスワード = uidJoe)
テストユーザーの mail	01PLM01@plm-lab.local / 01PLM02@plm-lab.localemailAddress NameID の元)
検索バインド	uid=idp-reader,ou=people,dc=example,dc=com (パスワード idp-reader・読取専用)
ApacheDS 管理者	uid=admin,ou=system (既定パスワード secret)
LDAP ポート	10389ApacheDS 既定。IdP 側は ldap://localhost:10389 で参照)

baseDN について (実機の判断) : ApacheDS のインストーラ版 (2.0.0.AM 系) は設定を config.ldif ではなく LDAP 内部 (ou=config) で保持し、dc=plm-lab,dc=local パーティションの手作業追加は難度が高い (7.4 参照)。本書では

既定パーティション **dc=example,dc=com** をそのまま流用する（学習・SSO 動作には影響しない）。WSL 版の dc=plm-lab,dc=local からの読み替え表は 7.4 に示す。mail は @plm-lab.local のまま（メールアドレスと baseDN は別物なので揃える必要はない）。

ポートの注意：ApacheDS の LDAP ポートは実機で **10389**（非特権ポート）。Directory Studio の New Connection では Port 入力欄の既定表示が **389** になることがあるが、実機の待受は 10389 なので **10389** に合わせる（`netstat -ano | findstr 10389` で確認）。WSL 版は 389 だったが、本書では 10389 を用い、フェーズ 5 の IdP 設定（ldapURL）も ldap://localhost:10389 とする。

7.2 ApacheDS のインストール

前提（**Java** を先に導入）：ApacheDS は Java で動作し、インストーラの途中で **JAVA_HOME** の入力を求められる。そのため **OpenJDK (Temurin 17)** を **ApacheDS** より前に導入しておく（\$5.6 参照）。インストーラで JAVA_HOME を聞かれたら、JDK のルート（例 C:\opt\jdk-17。直下に bin\java.exe があるフォルダ。bin は含めない）を指定する。

1. Apache Directory の公式サイト（<https://directory.apache.org/apacheds/download/download-windows.html>）から、**ApacheDS の Windows** インストーラ（.exe・最新安定版）を入手し、C:\lab\installers に保存。※ ApacheDS 本体はインストーラ（.exe）で導入する（zip 提供が無い。ため。zip 優先方針の例外）。
2. インストーラを管理者として実行。JAVA_HOME に上記 JDK を指定し、他は既定のまま進める（実機の導入先は C:\Program Files (x86)\ApacheDS、インスタンスは ...\instances\default）。導入すると **ApacheDS の Windows** サービス（既定インスタンス名 default）が登録される。
3. サービスを起動・確認：

```
Get-Service | Where-Object { $_.DisplayName -like "*ApacheDS*" }
Start-Service <サービス名> # 停止していれば開始（サービス名は環境で異なる）
# LDAP 待受ポートの確認（実機は 10389）
netstat -ano | findstr 10389
```

実機では LDAP は **10389** で待ち受ける（0.0.0.0:10389 LISTENING）。

7.3 Apache Directory Studio の導入と接続

1. 公式サイト（<https://directory.apache.org/studio/download/download-windows.html>）から **Apache Directory Studio (Windows 版・zip)** を入手し C:\lab\installers に保存、C:\opt\directory-studio に展開する（本書の zip 優先方針）。管理者 PowerShell：

```
Expand-Archive -Path "C:\lab\installers\ApacheDirectoryStudio-*.zip" -DestinationPath "C:\opt" -Force
# 展開されたフォルダ名を確認
Get-ChildItem C:\opt | Where-Object Name -like "*DirectoryStudio*"
```

C:\opt\directory-studio にリネーム (ApacheDirectoryStudio.exe が直下に来るように)

Rename-Item "C:\opt\ApacheDirectoryStudio" "C:\opt\directory-studio"

確認

Get-ChildItem C:\opt\directory-studio\ApacheDirectoryStudio.exe

2. 起動：zip 展開ではインストールされず、**Windows** スタートメニューにも登録されない。エクスプローラーまたはコマンドから **C:\opt\directory-studio\ApacheDirectoryStudio.exe** を実行して起動する（よく使う場合はショートカットを作成）。
 - Java が見つからず起動しない場合は、C:\opt\directory-studio\ApacheDirectoryStudio.ini に -vm と C:\opt\jdk-17\bin\javaw.exe の 2 行を追記する（通常は不要。JAVA_HOME 設定済みのため）。
3. 起動後、**LDAP** 接続を作成：メニュー **LDAP** → **New Connection**。
 - Network Parameter：Hostname localhost / Port **10389**（入力欄の既定表示が 389 の場合は 10389 に変更） / Encryption method **No encryption**（同一ホスト通信のため平文で可）
 - Connection name（任意）：管理作業用なので plm-lab-admin など用途が分かる名前を推奨
 - 次画面（Authentication）：Bind DN uid=admin,ou=system / Bind password secret
 - **Check Authentication** を押し、successful を確認してから **Finish**。接続が緑色になれば成功。
4. 左の **LDAP Browser** で、既定パーティション dc=example,dc=com や ou=system・ou=config が見えることを確認。

7.4 パーティション (baseDN) の方針：既定パーティション **dc=example,dc=com** を流用

当初は WSL 版に合わせ dc=plm-lab,dc=local パーティションを新規作成する想定だったが、実機のインストーラ版 **ApacheDS (2.0.0.AM 系)** では手作業での新規作成は現実的でないことが判明した。理由：

- このバージョンは設定を config.ldif ファイルではなく **LDAP 内部 (ou=config)** として保持する（実際 C:\Program Files (x86)\ApacheDS\instances\default\配下に config.ldif は存在せず、partitions\に example・system の DB がある）。パーティション追加は停止→設定エントリ (ads-partition…階層) を正確に追記→再起動が必要で、書式ミスでサービスが起動不能になるリスクがある。
- Directory Studio に ApacheDS を「サーバ」として登録していない（接続のみ）ため、**LDAP Servers** ビューの **Open Configuration** によるパーティション編集が使えない。

そこで本書は、**ApacheDS**の既定パーティション **dc=example,dc=com** をそのまま流用する。パーティション作成は不要で、その配下に **ou=people** とユーザーを作る。学習・SSO 動作には一切影響しない。

baseDN 読み替え表（当初の **plm-lab** 版 → 本書の既定流用版）

項目	当初（WSL 版踏襲）	本書（既定流用）
baseDN	dc=plm-lab,dc=local	dc=example,dc=com
ユーザー OU	ou=people,dc=plm-lab,dc=local	ou=people,dc=example,dc=com
テストユーザー	uid=01PLM01,ou=people,dc=plm-lab,dc=local	uid=01PLM01,ou=people,dc=example,dc=com
検索バインド	uid=idp-reader,ou=people,dc=plm-lab,dc=local	uid=idp-reader,ou=people,dc=example,dc=com
mail（変更なし）	01PLM01@plm-lab.local	01PLM01@plm-lab.local

どうしても **dc=plm-lab,dc=local** を作りたい場合は、**ApacheDS** の設定（**ou=config** の **ou=partitions**）にパーティション定義を追加し、コンテキストエントリ **dc=plm-lab,dc=local**）を投入する必要がある（バージョン依存・要サービス再起動）。本書では扱わない。

7.5 OU・テストユーザー・検索バインドの投入（LDIF）

Directory Studio で **LDAP** メニュー → **New LDIF File** を開き、以下を貼り付けて、エディタ右上の緑の ▶（**Execute LDIF**）で実行します（対象接続は管理者の **plm-lab-admin**）。**dc=example,dc=com** 自体は既存のため **LDIF** に含めず、その配下だけ作ります。パスワードは分かりやすさ優先の平文で記載します（**ApacheDS** が格納時に自動で **SSHA** ハッシュ化するため、WSL 版の **slappasswd** に相当する作業は不要）。

ユーザー OU（既存の **dc=example,dc=com** 配下に作成）

```
dn: ou=people,dc=example,dc=com
objectClass: top
objectClass: organizationalUnit
ou: people
```

テストユーザー 1（個人番号 01PLM01・Joe アカウント）

```
dn: uid=01PLM01,ou=people,dc=example,dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: 01PLM01
```

cn: Test User 01PLM01
sn: 01PLM01
mail: 01PLM01@plm-lab.local
userPassword: 01PLM01

テストユーザー 2 (個人番号 01PLM02・Joe アカウント)
dn: uid=01PLM02,ou=people,dc=example,dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: 01PLM02
cn: Test User 01PLM02
sn: 01PLM02
mail: 01PLM02@plm-lab.local
userPassword: 01PLM02

IdP 検索バインド用 (読取専用アカウント)
dn: uid=idp-reader,ou=people,dc=example,dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: idp-reader
cn: IdP Reader
sn: Reader
userPassword: idp-reader

実行前は未投入：LDIF タブ名が *LDIF … (先頭に *) の間は編集集中で、サーバへは未送信。右上の緑の  (**Execute LDIF**) を押して初めて反映される。成功すると下部 **Modification Logs** に #!RESULT OK と各 add が記録され、左ツリーの dc=example,dc=com を Reload すると ou=people 配下が現れる。userPassword は Entry editor で「SSHA hashed password」と表示され、平文が自動ハッシュ化されたことを確認できる。

objectClass の階層：inetOrgPerson は organizationalPerson → person → top を継承するため、上記のように4つを併記します。person の必須属性 cn・sn を必ず与えます (Directory Studio の GUI 追加でも同様に求められます)。

GUI で投入する場合：LDAP Browser で dc=example,dc=com を右クリック → **New Entry** → **Create entry from scratch** で ou=people (organizationalUnit) を作成 → 続けて ou=people の下に objectClass inetOrgPerson のエントリを作成、RDN

を uid=01PLM01、cn・sn・mail を入力し、最後に **New Attribute** で userPassword を追加。

7.6 動作確認 (検索バインド)

投入後、検索バインド (**idp-reader**) でテストユーザーを引けるかを確認します。これはフェーズ5でIdPが行う動作の先取り確認です。

方法1: **Directory Studio** で確認 - 新しい接続 (例: plm-lab-idp-reader) を Port 10389 / Encryption No encryption / Bind DN uid=idp-reader,ou=people,dc=example,dc=com / パスワード idp-reader で作成し、**Check Authentication** が successful になること。 - LDAP Browser で ou=people を展開し、uid=01PLM01・01PLM02 が見えること (mail 属性に 01PLM01@plm-lab.local 等が入っていること)。

方法2: 検索で確認 (推奨) Directory Studio の検索機能で、検索ベース ou=people,dc=example,dc=com、フィルタ (uid=01PLM01) を実行し、1件返ることを確認する。具体的な操作:

1. LDAP Browser で ou=people,dc=example,dc=com を選択。
2. 右クリック - **New Search...** (またはメニュー **Search - New Search...**、Ctrl+H)。
3. Search ダイアログで設定:
 - **Search Base**: ou=people,dc=example,dc=com (手順1で選択していれば入力済み。空なら Browse... で選択)
 - **Filter**: (uid=01PLM01) (括弧を含めて入力)
 - **Returning Attributes**: 空でよい (mail uid cn と指定してもよい)
 - **Scope**: Subtree (既定)
4. **Search** ボタンをクリック。
5. 検索結果ビューに uid=01PLM01,ou=people,dc=example,dc=com が1件表示されれば成功 (行を選ぶと右の Entry editor で mail 等を確認できる)。

簡易的には、LDAP Browser 上部のフィルタ入力欄に (uid=01PLM01) を入力して Enter でも絞り込める。検索結果として明示的に確認するなら上記の New Search が確実。**plm-lab-idp-reader** 接続 (**idp-reader** で **bind**) で実行すると、**IdP** が実際に行う検索を模擬できる。

#	確認内容	期待結果
1	ApacheDS サービス稼働	ApacheDS サービスが実行中、LDAP ポート 10389 で待受
2	管理者接続	uid=admin,ou=system / secret で接続可
3	ベース DN	既定パーティション dc=example,dc=com を使用
4	エントリ投入	ou=people,dc=example,dc=com 配下に

#	確認内容	期待結果
5	検索バインド	01PLM01 / 01PLM02 / idp-reader uid=idp-reader,ou=people,dc=example,dc=com で bind (Check Authentication successful) し、(uid=01PLM01) が1件返る
6	mail 属性	01PLM01 の mail が 01PLM01@plm-lab.local
7	パスワード格納	userPassword が「SSHA hashed password」 (平文が自動ハッシュ化)

フェーズ5のIdPは、このidp-readerでLDAPに検索バインドし、ログインユーザーのuidで認証、mailをNameID (emailAddress形式) の元として取得します。フェーズ5のIdP設定でそのまま使う値：**LDAP URL ldap://localhost:10389/**検索ベース **ou=people,dc=example,dc=com** / 検索バインド **uid=idp-reader,ou=people,dc=example,dc=com** / パスワード **idp-reader** / フィルタ (**uid={user}**) / NameID (mail)。

8. フェーズ4 : OpenJDK + Tomcat (Windows サービス)

目的：Shibboleth IdP (フェーズ5) を載せる土台として、Tomcat 10.1 を Windows サービスとして稼働させる。OpenJDK (Temurin 17) は \$5.6 で導入済みのため、本フェーズは Tomcat に集中する。まず HTTP 8080 で起動確認し、HTTPS 8443 化はフェーズ6で行う。

前提：java -version が 17 を返し、JAVA_HOME=C:\opt\jdk-17 が設定済みであること (\$5.6)。IdP 5 は **Tomcat 10.1** 系が必須 (9 系や 11 は不可。Jakarta 名前空間)。

8.1 Tomcat 10.1 の入手と展開 (zip)

1. Apache Tomcat 公式 (<https://tomcat.apache.org/download-10.cgi>) から、**Tomcat 10.1.x** の「**Core**」 **zip** (例 apache-tomcat-10.1.xx-windows-x64.zip または apache-tomcat-10.1.xx.zip) を入手し C:\lab\installers へ。
2. C:\opt\tomcat に展開 (zip 内のトップフォルダを C:\opt\tomcat に合わせる)。
管理者 PowerShell :

```
Expand-Archive -Path "C:\lab\installers\apache-tomcat-10.1.*.zip" -DestinationPath "C:\opt" -Force
```

展開されたフォルダ名を確認 (例 : apache-tomcat-10.1.56)

```
Get-ChildItem C:\opt | Where-Object Name -like "apache-tomcat-*
```

C:\opt\tomcat にリネーム (bin/conf/lib など直下に来るように)

```
Rename-Item "C:\opt\apache-tomcat-10.1.xx" "C:\opt\tomcat"
```

確認: bin\startup.bat 等が見えること

```
Get-ChildItem C:\opt\tomcat\bin\startup.bat, C:\opt\tomcat\conf\server.xml
```

8.2 環境変数 (CATALINA_HOME) の設定

Tomcat の場所を示す CATALINA_HOME をシステム環境変数に設定します (サービス化・起動スクリプトが参照)。管理者 PowerShell :

```
[Environment]::SetEnvironmentVariable("CATALINA_HOME", "C:\opt\tomcat", "Machine")
```

設定後は新しい **PowerShell** / コマンドプロンプトを開き直す (既存セッションには反映されない)。JAVA_HOME は \$5.6 で設定済みのため、Tomcat のサービスも JVM を見つけられる。

8.3 Windows サービスとして登録 (service.bat)

zip 版には bin\service.bat (サービス登録スクリプト) が含まれます。管理者コマンドプロンプト ☐ PowerShell ではなく cmd を推奨。service.bat はバッチのため) で実行します。

```
cd /d C:\opt\tomcat\bin
service.bat install
```

- 成功すると、サービス名 **Tomcat10** (既定。サービスの表示名は実機で「**Apache Tomcat 10.1 Tomcat10**」となる) が登録されます (service.bat install <名前> で任意名も可)。
- UAC が有効な場合、Tomcat10.exe 起動時に追加の権限を求められることがあります (管理者で実行していれば通過)。

JVM (Java) をサービスに確実に認識させる: サービスが JVM を見つけられないと起動に失敗します。GUI 設定ツール tomcat10w で確認・調整できます。

```
C:\opt\tomcat\bin\tomcat10w.exe //ES//Tomcat10
```

- 開いたダイアログの **Java** タブで、**Java Virtual Machine** が C:\opt\jdk-17\bin\server\jvm.dll (または ...\bin\jvm.dll) を指しているか確認。空欄や誤りがあれば設定。
- Startup type** を Automatic (自動起動) にしておくと、Windows 起動時に Tomcat も上がる (再起動堅牢性)。

service.bat install 実行時に JAVA_HOME が正しく通っていれば、JVM は自動設定されることが多いです。起動に失敗する場合はまずこの tomcat10w の Java タブを確認します。

8.4 起動と動作確認 (8080)

```
Start-Service Tomcat10
```

```
Get-Service Tomcat10    # Status が Running
```

HTTP 応答 (既定コネクタは 8080)

Invoke-WebRequest http://localhost:8080/ -UseBasicParsing | Select-Object StatusCode

待受ポート確認

netstat -ano | findstr 8080

- ブラウザで http://localhost:8080/ を開き、Tomcat の初期ページが表示されれば成功。
- ログは C:\opt\tomcat\logs\catalina.*.log) で確認できます。

補足 (8080 の公開範囲) : 本構成では IdP へのブラウザアクセスはフェーズ 6 で HTTPS 8443 経由にするため、8080 は最終的に localhost 限定でも構いません (server.xml の HTTP コネクタに address="127.0.0.1" を付けて絞れる)。フェーズ 4 の時点では既定 (全インターフェース) で確認して問題ありません。

8.5 動作確認チェックリスト (フェーズ 4)

#	確認内容	コマンド / 方法	期待結果
1	Java	java -version	17.x
2	Tomcat 展開	C:\opt\tomcat\bin\startup.bat 等の存在	直下に bin/conf/lib
3	環境変数	echo %CATALINA_HOME% (新規 cmd)	C:\opt\tomcat
4	サービス登録	Get-Service Tomcat10	サービスが存在
5	サービス稼働	Get-Service Tomcat10	Status: Running
6	HTTP 応答	http://localhost:8080/	初期ページ (200)
7	自動起動	tomcat10w の Startup type	Automatic

すべて確認できれば、フェーズ 4 は完了です。次はフェーズ 5 Shibboleth IdP 5 の導入・LDAP 認証・emailAddress 形式 NameID の準備) です。

9. フェーズ 5 : Shibboleth IdP 5 (テスト IdP)

目的 : Windows 上の Tomcat に Shibboleth IdP 5 を導入し、ApacheDS(LDAP) で認証、mail 属性を emailAddress 形式の NameID の元として扱う準備までを行う。将来この IdP は Entra ID に差し替え可能 (学習用テスト IdP)。

WSL 版との違い : install.sh → install.bat、Linux パス → Windows パス、systemctl restart → Tomcat サービス再起動。SAML の設計 (LDAP 認証・NameID) は共通。NameID 形式は WSL 版の unspecified から emailAddress に変更し、元属性を uid → mail にする (顧客 Entra の形式に合わせる)。

前提：フェーズ4でTomcatが8080で起動確認済み。フェーズ3のLDAP引き継ぎ値 (ldap://localhost:10389/ou=people,dc=example,dc=com/uid=idp-reader,ou=people,dc=example,dc=com/idp-reader/(uid={user})/NameIDの元mail) を使う。

9.1 IdP 5 の入手と展開

1. Shibboleth 公式 (<https://shibboleth.net/downloads/identity-provider/latest5/>) から **IdP 5** の **zip** Windows は zip 推奨。Windows 改行) を入手し C:\lab\installers へ。
 - latest5/ にはその時点の最新版だけが置かれる。实在版のファイル名を確認してから取得する (例：ブラウザで上記 URL を開き、shibboleth-identity-provider-5.x.y.zip を確認)。
2. 任意の場所 (例 C:\lab) に展開。展開先は導入後は不要。

```
Expand-Archive -Path "C:\lab\installers\shibboleth-identity-provider-5.*.zip" -DestinationPath "C:\lab" -Force
```

```
Get-ChildItem C:\lab | Where-Object Name -like "shibboleth-identity-provider-5*"
```

展開フォルダのリネームは不要 (Java/Tomcat と異なる点)。IdP の展開フォルダ (例 C:\lab\shibboleth-identity-provider-5.2.3) は **install.bat** を実行するための一時的な作業場所にすぎず、恒久的に使うのは install.bat が配置する **C:\opt\shibboleth-idp** (idp.home。バージョン番号なし)。Java/Tomcat をリネームしたのは、展開先がそのまま JAVA_HOME/CATALINA_HOME という恒久パスになるため。むしろ展開フォルダ名にバージョンが残っていると、どの版から導入したかの記録になり、将来のアップグレード時に新旧が混ざらない。

9.2 install.bat による導入

展開したディストリビューションフォルダに入り、bin\install.bat を管理者コマンドプロンプトで実行します (対話式)。

```
cd /d C:\lab\shibboleth-identity-provider-5.x.y
bin\install.bat
```

対話 (プロンプト) での入力、実機 (IdP 5.2.3) では 4 項目のみ：

質問	入力値
Installation Directory (idp.home)	C:\opt\shibboleth-idp
Host Name	idp.plm-lab.local
SAML EntityID	https://idp.plm-lab.local/idp/shibboleth
Attribute Scope	plm-lab.local

⚠ **Host Name** の既定値に注意：プロンプトが Host Name: [sp.plm-lab.local] ? のように **SP** のホスト名を既定値として表示することがある（マシン名等から推測されるため）。そのまま **Enter** せず、必ず **idp.plm-lab.local** を明示入力する（誤ると IdP が SP のホスト名で構築されてしまう）。

Keystore Password / Sealer Password は聞かれない：IdP 5.2.3 では、鍵とパスワードが自動生成され `credentials\secrets.properties` に格納される（対話で入力を求められない）。したがって §3.1 のとおり、これらに `changeit` 等を設定する作業は不要。

なぜ自動生成なのか：これらのパスワードは人間がログインに使うものではなく、**IdP** が自分自身の鍵ファイル（署名・暗号鍵、**Sealer** 鍵）を開くための内部的な資格情報である。人間に決めさせると弱い値（`changeit` など）になりがちのため、IdP 5 ではインストーラが強いランダム値を自動生成して `secrets.properties` に書き込む方式が変わった（IdP 3/4 世代は対話で入力を求めていたため、古い手順書やネット上の情報には「Keystore Password を入力」と書かれていることが多い）。

- 既定値からの変更を心配する必要はない：固定の既定値ではなく、インストールのたびに異なるランダム値が生成されるため、すでに強い値になっている。変更は不要。
- パスワードだけを書き換えてはいけない：パスワードは「鍵ファイルを開くための鍵」なので、文字列だけ変えると鍵ファイルと不整合を起こし **IdP** が起動しなくなる。変更したい場合は鍵ファイル自体を作り直す必要がある（**Sealer** は `bin\seckeygen.bat` 等）。通常は自動生成のまま使う。
- PFX** のパスワード（**changeit**）とは別物：§3.1 #7 の「TLS 証明書 PFX[idp/sp] = changeit」は引き続き有効。**PFX** は自分で **openssl** で作った TLS 証明書 (a) のファイルなので、パスワードも自分で決め、Tomcat[server.xml] や IIS へのインポート時に指定する。一方、キーストア/**Sealer** は **IdP** が自分で作った **SAML** 鍵 (b) 用で、IdP が自動生成・管理する。「自分が作ったファイルのパスワードは自分で決める／IdP が作ったファイルのパスワードは IdP が管理する」と整理すると分かりやすい（証明書 (a)(b) の違い = §6.3 とも対応）。
- 導入すると、署名・暗号化鍵(b)[idp-signing / idp-encryption / backchannel / Sealer, keySize=3072] が自動生成され、`C:\opt\shibboleth-idp\` に配置、`metadata\idp-metadata.xml` が作成され、`war\idp.war` がビルドされる。
- RIPEMD-160 などの INFO ログは無害（SAML では使わない）。

`install.bat` が `JAVA` を見つけられない場合は、`JAVA_HOME=C:\opt\jdk-17` §5.6) が設定され、新しいコマンドプロンプトで実行しているか確認する。

9.3 Tomcat への配置 (context フラグメント)

Tomcat に IdP の `war` を認識させる **context** フラグメント `idp.xml` を配置します。

- フォルダを作成（無ければ）：`C:\opt\tomcat\conf\Catalina\localhost\`
- `idp.xml` を作成（管理者権限。昇格 PowerShell 例）：

```
New-Item -ItemType Directory -Force -Path "C:\opt\tomcat\conf\Catalina\localhost" | Out-Null
Set-Content -Path "C:\opt\tomcat\conf\Catalina\localhost\idp.xml" -Encoding ASCII -Value '<Context docBase="C:/opt/shibboleth-idp/war/idp.war" privileged="true" antiResourceLocking="false" swallowOutput="true" />'
```

docBase のパス区切りは / (スラッシュ) で記述する (Tomcat の context では Windows でも / が無難)。

JSTL の追加 (必須) : IdP 5.2.3 では、状態ページ /idp/profile/status などが JSP + JSTL で描画されるため、**JSTL** を **war** に追加しないと **ClassNotFoundException: jakarta.servlet.jsp.jstl.core.Config - ServletException** になる (メタデータ /idp/shibboleth は JSTL 不要のため返るが、status は失敗する)。次の手順で **API** と実装 (**Glassfish**) の **2 jar** を overlay に置き、build.bat で war を再ビルドする。

```
$lib = "C:\opt\shibboleth-idp\edit-webapp\WEB-INF\lib"
New-Item -ItemType Directory -Force -Path $lib | Out-Null
# JSTL API
Invoke-WebRequest -UseBasicParsing -Uri "https://repo1.maven.org/maven2/jakarta/servlet/jsp/jstl/jakarta.servlet.jsp.jstl-api/3.0.0/jakarta.servlet.jsp.jstl-api-3.0.0.jar" -OutFile "$lib\jakarta.servlet.jsp.jstl-api-3.0.0.jar"
# JSTL 実装 (Glassfish) ※ 実装が無いと Config クラスが見つからずエラーになる
Invoke-WebRequest -UseBasicParsing -Uri "https://repo1.maven.org/maven2/org/glassfish/web/jakarta.servlet.jsp.jstl/3.0.1/jakarta.servlet.jsp.jstl-3.0.1.jar" -OutFile "$lib\jakarta.servlet.jsp.jstl-3.0.1.jar"

cd /d C:\opt\shibboleth-idp\bin
build.bat
```

Rebuilding ...\war\idp.war / Overlay from ...\edit-webapp / Creating war file ...\war\idp.war が出れば成功。オフライン環境では別端末で 2 jar を取得し edit-webapp\WEB-INF\lib\ にコピーしてから build.bat する。

9.4 LDAP 認証の設定 (ldap.properties)

 **【最重要】** バックアップの取り方 (ここを誤ると認証が失敗する) : 設定ファイルを編集する前にバックアップを取る場合、**conf** および **credentials** 配下に拡張子 **.properties** のままコピーを置いてはいけない。IdP はこれらのディレクトリの **.properties** をすべて読み込むため、編集前の既定値 (**uid=myservice,ou=system / dc=example,dc=org / myServicePassword**) が読み込まれて編集後の値と競合し、ログイン時に **PoolExhaustedException: Pool is empty and connection creation failed** で失敗する (ApacheDS が正常稼働し、エントリもパスワードも正しくても発生する)。

- 危険な例：Windows の「コピー」で作られる **ldap - Copy.properties**（拡張子が .properties のまま = 読み込まれる）。
- 正しい例：**ldap.properties.orig** のように拡張子を変える（.orig/.bak）、または **C:\lab\idp-backup** など **IdP** の外へ置く。
- 起動時に **WARN ... Duplicate properties were detected: ...** が出たら、このパターンを疑い、conf/credentials 配下に余計な .properties が無いか確認する（\$ 9.6・付録 D）。

C:\opt\shibboleth-idp\conf\ldap.properties を編集し、フェーズ 3 の値に合わせます（平文 **LDAP・10389・dc=example,dc=com**）。主な項目：

```
idp.authn.LDAP.authenticator      = bindSearchAuthenticator
idp.authn.LDAP.ldapURL            = ldap://localhost:10389
idp.authn.LDAP.useStartTLS        = false
idp.authn.LDAP.useSSL             = false
idp.authn.LDAP.baseDN             = ou=people,dc=example,dc=com
idp.authn.LDAP.subtreeSearch      = true
idp.authn.LDAP.userFilter         = (uid={user})
idp.authn.LDAP.bindDN             = uid=idp-reader,ou=people,dc=example,dc=com
idp.authn.LDAP.dnFormat           = uid=%s,ou=people,dc=example,dc=com
# 返す属性に mail を含める (NameID の元)
idp.authn.LDAP.returnAttributes   = mail,uid
```

- 検索バインドのパスワードは credentials\secrets.properties に集約します（WSL 版と同様、二重定義による WARN を避ける）。C:\opt\shibboleth-idp\credentials\secrets.properties の該当行：

```
idp.authn.LDAP.bindDNCredential = idp-reader
```

ldap.properties 側に idp.authn.LDAP.bindDNCredential の行があればコメントアウトして重複を避ける。

-  **trustCertificates**／**trustStore** は必ずコメントアウトする（必須）：インストール直後は次の 2 行が有効になっている。平文 LDAP（useStartTLS=false/useSSL=false）では本来不要だが、有効のままだと存在しないファイル（**ldap-server.crt**／**ldap-server.truststore**）を参照して **LDAP** 接続プールの初期化に失敗し得る。

```
#idp.authn.LDAP.trustCertificates = %{idp.home}/credentials/ldap-server.crt
#idp.authn.LDAP.trustStore       = %{idp.home}/credentials/ldap-server.truststore
```

なお idp.attribute.resolver.LDAP.trustCertificates = %{idp.authn.LDAP.trustCertificates:undefined} の行は変更不要 (authn 側をコメントアウトすれば undefined に解決され無効化される)。

- 属性解決 (attribute-resolver) でも同じ LDAP を参照する場合は idp.attribute.resolver.LDAP.* を同様に設定するが、本書は NameID の元 mail を authn 側の returnAttributes で取得する構成を基本とする (フェーズ 10 で属性解放と NameID 生成を確定)。

9.5 emailAddress 形式 NameID の準備 (saml-nameid.xml)

mail 属性を **emailAddress** 形式の **NameID** として発行する定義を、C:\opt\shibboleth-idp\conf\saml-nameid.xml の <util:list id="shibboleth.SAML2NameIDGenerators"> の内側に追加します (詳細な解放設定はフェーズ 10 で仕上げる。ここでは生成器の準備まで)。

```
<bean parent="shibboleth.SAML2AttributeSourcedGenerator"
  p:omitQualifiers="true"
  p:format="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress"
  p:attributeSourceIds="#{'mail'}" />
```

これにより、ログインユーザーの mail (例 01PLM01@plm-lab.local) が emailAddress 形式の NameID として発行できる状態になる。SP 側での REMOTE_USER へのマッピングはフェーズ 10 で行う。

注意 (重複させない) : 既存のコメントアウト例を有効化する場合は、それとは別に同じ bean を追記しないこと。shibboleth.SAML2NameIDGenerators の中に同一の SAML2AttributeSourcedGenerator が 2 つ入っていると意図しない重複になる (1 つだけにする)。SAML1 側 (shibboleth.SAML1NameIDGenerators) は本構成では不要。

9.6 起動と動作確認

Restart-Service Tomcat10

Start-Sleep -Seconds 20

ステータス (テキストが返る)

Invoke-WebRequest http://localhost:8080/idp/profile/status -UseBasicParsing | Select-Object -ExpandProperty Content

メタデータ (<EntityDescriptor...> が返る)

(Invoke-WebRequest http://localhost:8080/idp/shibboleth -UseBasicParsing).Content.Substring(0,300)

ログの確認 (**PowerShell**) : idp-process.log は追記式 (過去の起動分も残る) のため、最新の起動ブロックだけを見る。IdP は起動のたびに Shibboleth IdP Version の行を出すので、これを目印にする。

最新の起動以降の ERROR / WARN を抽出（何も返らなければ正常）

```
$log = "C:\opt\shibboleth-idp\logs\idp-process.log"
```

```
$lines = Get-Content $log
```

```
$start = ($lines | Select-String 'Shibboleth IdP Version' | Select-Object -Last 1).LineNumber
```

```
$lines[($start-1)..($lines.Count-1)] | Select-String 'ERROR|WARN'
```

簡易確認（末尾のみ）

```
Get-Content $log -Tail 50 | Select-String 'ERROR|Shibboleth IdP Version'
```

トラブル時：リアルタイムで追う（Ctrl+C で停止）

```
Get-Content $log -Wait -Tail 20
```

メッセージの判断基準（ERROR だから即異常、ではない。内容で判断する）：

メッセージ	判断	対処
ERROR ... unable to find resource 'status.vm'	無害（status ページはフォールバックで正常に表示される）	不要
INFO ... Algorithm failed runtime support check ... ripemd160	無害（SAML では使わない）	不要
WARN ... Duplicate properties were detected: idp.*.LDAP.*	⚠ 要確認	conf / credentials 配下に余計な .properties （バックアップコピー等）が無い か確認（\$9.4・付録 D）。放置 するとログインが Pool is empty... で失敗する
WARN ... Duplicate Definition 'mail'	要対処	attribute-resolver.xml の mail 定義を 1 つにする（\$14.1）
ERROR ... ClassNotFoundException: jakarta.servlet.jsp.jstl.core.Config	要対処	JSTL 2 jar を追加し build.bat（\$9.3）

- install.bat 導入直後に bin\status.bat を実行すると、既定で http://localhost/idp/status（80 番）を見にいき「Connection refused」になることがあるが、これは想定どおり（本構成は 8080 / のちに 8443）。確認は上記 8080 の URL で行う。

9.7 動作確認チェックリスト（フェーズ 5）

#	確認内容	期待結果
1	IdP 導入	C:\opt\shibboleth-idp\ に conf/credentials/metadata/war が存在

#	確認内容	期待結果
2	context 配置	C:\opt\tomcat\conf\Catalina\localhost\idp.xml が存在
2b	JSTL 追加	edit-webapp\WEB-INF\lib に JSTL の API + 実装 2 jar があり build.bat 済み
3	Tomcat 起動	Tomcat10 が Running
4	ステータス	http://localhost:8080/idp/profile/status が環境情報を返す
5	メタデータ	http://localhost:8080/idp/shibboleth が <EntityDescriptor> を返す
6	LDAP 設定	ldap.properties が 10389 / dc=example,dc=com / idp-reader、returnAttributes に mail
7	NameID 準備	saml-nameid.xml に emailAddress の SAML2AttributeSourcedGenerator (mail)
8	ログ	idp-process.log に致命的 ERROR ・ 重複 WARN が無い

すべて確認できれば、フェーズ5は完了です。次はフェーズ6 (Tomcat を直接 HTTPS 8443 で公開) です。

10. フェーズ6 : Tomcat 直 HTTPS (8443) 公開

目的：ブラウザから IdP へ **HTTPS (8443)** で到達できるようにする。WSL 版では前段に Apache を置いて TLS を終端したが、本構成では **Tomcat** 自身に **HTTPS** コネクタを持たせて **Apache** を省略する。証明書はフェーズ2で作成した idp.pfx (TLS/HTTPS サーバ証明書a) ・ パスワード changeit) を使う。

WSL 版との違い：Apache HTTPD の導入 ・ リバースプロキシ設定 ・ mirrored ネットワーク ・ Listen 80/443 無効化がすべて不要。Tomcat の server.xml に SSL コネクタを1つ追加するだけ。

10.1 証明書 (idp.pfx) を Tomcat に配置

フェーズ2で作成した C:\lab\ca\idp.pfx を Tomcat の conf に配置します (管理者 Powershell)。

```
Copy-Item "C:\lab\ca\idp.pfx" "C:\opt\tomcat\conf\idp.pfx" -Force
Get-Item "C:\opt\tomcat\conf\idp.pfx"
```

idp.pfx (は idp.crt + idp.key + rootCA.crt を含む PKCS12 形式 SAN=idp.plm-lab.local) 。
パスワードは changeit (3.1) 。

10.2 server.xml に HTTPS 8443 コネクタを追加

C:\opt\tomcat\conf\server.xml を管理者権限で編集します。既存の 8080 HTTP コネクタ (<Connector port="8080" .../>) の近くに、次の HTTPS 8443 コネクタを追加します (Tomcat 10.1 は **SSLHostConfig + Certificate** 方式) 。

```
<Connector port="8443" protocol="org.apache.coyote.http11.Http11NioProtocol"
    maxThreads="150" SSLEnabled="true" scheme="https" secure="true">
  <SSLHostConfig>
    <Certificate certificateKeystoreFile="conf/idp.pfx"
      certificateKeystorePassword="changeit"
      certificateKeystoreType="PKCS12"
      type="RSA" />
  </SSLHostConfig>
</Connector>
```

- certificateKeystoreFile="conf/idp.pfx" は CATALINA_BASE (= C:\opt\tomcat) からの相対で解決される。絶対パスにする場合は C:/opt/tomcat/conf/idp.pfx (スラッシュ) で書く。
- パスワードに &・<・> を含む場合は XML エスケープが必要 (今回の changeit は不要) 。

10.3 8080 を localhost 限定にする (任意・推奨)

ブラウザは 8443 経由で IdP にアクセスするため、平文の 8080 は localhost だけに絞っておくと安全です。既存の 8080 コネクタに address="127.0.0.1" を追加します。

```
<Connector port="8080" protocol="HTTP/1.1"
    address="127.0.0.1"
    connectionTimeout="20000"
    redirectPort="8443" />
```

redirectPort を 8443 にしておくと、機密制約でリダイレクトが必要な場合に HTTPS へ誘導される。必須ではない。

この設定は外部からの **SSO** に影響しない (実機確認済み) : SSO の経路は **443 (SP/IIS) → 8443 (IdP/Tomcat)** であり、**8080** は経路に含まれない。したがって 8080 を localhost に絞っても、Hyper-V ホストや LAN の別 PC からの SSO は問題なく成立する (実機で確認済み) 。むしろ、パスワードを入力する **IdP** のログイン画面が平文 **HTTP** で外部に露出しないため、セキュリティ上も望ましい。ゲスト内からの診断 (http://localhost:8080/idp/profile/status) は引き続き利用できる。

10.4 反映と動作確認

Restart-Service Tomcat10

Start-Sleep -Seconds 20

8443 が待受

netstat -ano | findstr 8443

HTTPS の確認は【メタデータ】で行う（アクセス制御の影響を受けない）

Invoke-WebRequest https://idp.plm-lab.local:8443/idp/shibboleth -UseBasicParsing | Select-Object StatusCode

status（診断用）は localhost で確認する

Invoke-WebRequest http://localhost:8080/idp/profile/status -UseBasicParsing | Select-Object -ExpandProperty Content

- メタデータ (idp/shibboleth) が証明書エラーを出さずに **200** を返せば、**8443** 区 HTTP 公開・rootCA 信頼 (\$6.5) ・SAN (idp.plm-lab.local) がすべて正しいと確認できる。
- ゲスト Windows のブラウザで https://idp.plm-lab.local:8443/idp/shibboleth を開き、鍵マーク（証明書警告なし）であることを確認。
- 起動に失敗する場合は C:\opt\tomcat\logs\catalina.*.log を確認（多くは certificateKeystoreFile のパス誤り、certificateKeystorePassword 不一致、certificateKeystoreType の指定漏れ）。

⚠ hosts を実 IP にした場合、https://idp.plm-lab.local:8443/idp/profile/status は Web Login Service - Access Denied を返す（想定内・異常ではない）。IdP の管理系エンドポイント (status) は conf/access-control.xml の AccessByIPAddress により既定で 127.0.0.1 / ::1 からのみ許可されており、実 IP からのアクセスは拒否されるため TLS・接続・証明書は正常に IdP から返ってくるが、SSO 本体 (/idp/profile/SAML2/...) には影響しない。status を見るときは http://localhost:8080/idp/profile/status (localhost) で確認する。どうしても外部から status を見たい場合は conf/access-control.xml の allowedRanges にサブネットを助ける（管理用エンドポイントのためむやみに広げない）: xml <bean id="AccessByIPAddress" parent="shibboleth.IPRangeAccessControl" p:allowedRanges="#{'127.0.0.1/32', '::1/128', '192.168.137.0/24'}'" />

証明書警告が出る場合：rootCA が「信頼されたルート証明機関」に入っているか (\$6.5、certlm.msc)、アクセス URL のホスト名が idp.plm-lab.local (SAN と一致) か、hosts 解決を確認。

10.5 動作確認チェックリスト（フェーズ 6）

#	確認内容	期待結果
1	証明書配置	C:\opt\tomcat\conf\idp.pfx が存在
2	server.xml	8443 の SSLHostConfig + Certificate コネクタを追加

#	確認内容	期待結果
3	サービス起動	Tomcat10 がRunning(8443 コネクト成功)
4	待受	netstat で8443 がLISTENING
5	HTTPS 応答	https://idp.plm-lab.local:8443/idp/shibboleth (メタデータ) が 200 (証明書エラーなし)。※ status は実 I P 運用だと Access Denied になるため、確認はメタデータで行う
6	ブラウザ	メタデータURL が鍵マークで表示 (警告なし)
7	8080	(任意) address="127.0.0.1" でlocal host 限定

すべて確認できれば、フェーズ6は完了です。WSL 版で必要だった Apache 前段が不要になり、IdP がブラウザから HTTPS で到達可能になりました。次はフェーズ7(IIS の構築・SP の保護対象サイトと 443/TLS) です。

11. フェーズ7 : IIS (SP の保護対象サイト・443/TLS)

目的 : ゲスト Windows に IIS を導入し、保護対象となる PLM 相当のサイト (認証後に識別子 = REMOTE_USER を表示する確認ページ) を 443/HTTPS で用意する。フェーズ2の **sp** 証明書 (a) sp.pfx) を初めて使う。フェーズ8でShibboleth SP(IIS ネイティブモジュール) を組み込む土台。

WSL 版との違い : IIS も IdP(Tomcat) も同一の Windows 上にあり、sp.plm-lab.local-127.0.0.1 で素直に届くため、mirrored ネットワークの検討は不要。sp.pfx は WSL の \\wsl\$ 経由ではなく、フェーズ2で入れた **C:\lab\ca\sp.pfx** から直接取り込む。

11.1 IIS の導入 (Windows の機能)

フェーズ8のShibboleth SPはIIS モジュールとして動くため、ISAPI 拡張/フィルタを含めて有効化します。確認ページ用に古典 **ASP** も入れます。管理者 **PowerShell** で :

```
Enable-WindowsOptionalFeature -Online -All -FeatureName `
  IIS-WebServerRole, IIS-WebServer, IIS-CommonHttpFeatures, IIS-StaticContent, `
  IIS-DefaultDocument, IIS-ISAPIExtensions, IIS-ISAPIFilter, IIS-ASP, `
  IIS-RequestFiltering, IIS-WebServerManagementTools, IIS-ManagementConsole
```

GUI の場合 : 「コントロールパネル」→「プログラムと Windows の機能の有効化または無効化」→「インターネットインフォメーションサービス」で「World Wide Web サービス」アプリケーション開発機能

ISAPI 拡張機能 / ISAPI フィルター / ASP」と「Web 管理ツール IIS 管理コンソール」を有効化

確認: ブラウザで `http://localhost/` に IIS の既定ページが表示される。

11.2 hosts (確認)

`sp.plm-lab.local` はフェーズ 2 §6.2 で登録済み。未登録なら追加します。

`ping -n 1 sp.plm-lab.local` # 127.0.0.1 に解決されること

11.3 sp 証明書 (a) を Windows に取り込む

フェーズ 2 で作成した `C:\lab\ca\sp.pfx` を Windows の証明書ストア (ローカルコンピューター\個人) へ取り込みます。パスワードは §3.1 のとおり `changeit`。

```
Import-PfxCertificate -FilePath "C:\lab\ca\sp.pfx" `
-CertStoreLocation Cert:\LocalMachine\My `
-Password (ConvertTo-SecureString "changeit" -AsPlainText -Force)
```

rootCA はフェーズ 2 §6.5 で「信頼されたルート証明機関」に登録済みのため、ブラウザが sp 証明書を信頼できます。取り込んだ証明書の Thumbprint / Subject (CN=sp.plm-lab.local) が表示されれば成功。

11.4 既定サイトに 443/HTTPS バインドを追加

```
Import-Module WebAdministration
```

```
$cert = Get-ChildItem Cert:\LocalMachine\My |
Where-Object { $_.Subject -like "*CN=sp.plm-lab.local*" } | Select-Object -First 1
New-WebBinding -Name "Default Web Site" -Protocol https -Port 443
New-Item -Path "IIS:\SslBindings\0.0.0.0!443" -Value $cert
```

GUI の場合: IIS マネージャー → 「Default Web Site」→ 右側「バインド」→ 追加 → 種類 https / ポート 443 / SSL 証明書 sp.plm-lab.local を選択

11.5 確認用ページ (REMOTE_USER 表示)

`C:\inetpub\wwwroot\whoami.asp` を作成します。昇格したプロセス (管理者 PowerShell) で作るのが確実です。

```
$asp = '@'
<%@ Language="VBScript" %>
<%
Response.CodePage = 65001
Response.Charset = "utf-8"
%>
<html><body>
<h2>Authentication Test</h2>
```

```
REMOTE_USER = [<%= Request.ServerVariables("REMOTE_USER") %>]<br>
AUTH_TYPE = [<%= Request.ServerVariables("AUTH_TYPE") %>]
</body></html>
```

'@

Set-Content -Path "C:\inetpub\wwwroot\whoami.asp" -Value \$asp -Encoding UTF8

この段階ではSP未導入のためREMOTE_USERは空[]で表示されます。これは正常で、フェーズ8～10でSAML認証が通ると、ここに識別子（メール形式 **01PLM01@plm-lab.local**）が入ります。

補足1（ファイル作成権限・OS差異）：C:\inetpub\wwwrootへの書き込みは昇格したプロセスで行う。ビルトインAdministratorの挙動はOSで異なり、**Windows Server 2016**は既定で管理者承認モードが無効のためエクスプローラーの「新規作成」でwwwrootに直接書けるが、**Windows 11**クライアントは承認モードが効きエクスプローラーが標準トークン動作のため書けない（UACスライダーを最下段にしても変わらない）。承認モード無効はセキュリティ低下のため非推奨で昇格プロセス（管理者PowerShell／「管理者として実行」したメモ帳）での作成を推奨。組織の検証環境（Server系）ではこの制限自体が起きにくい。

補足2（文字コード）：日本語を含む古典ASPを英語ロケールのIISで表示すると文字化けすることがある。上記のように先頭でResponse.CodePage=65001／Response.Charset="utf-8"を指定し、BOM無しUTF-8で保存すれば回避できる。確認用途では見出しを英語表記（例：Authentication Test）にしておくのが無難。

11.6 動作確認

ゲスト Windows のブラウザで確認します。

#	確認内容	期待結果
1	IIS 既定ページ	http://localhost/ が表示される
2	HTTPS バインド	https://sp.plm-lab.local/ が鍵マーク（証明書エラーなし）で表示
3	ASP 動作	https://sp.plm-lab.local/whoami.asp が表示される
4	REMOTE_USER	同ページで REMOTE_USER = []（空。SP未導入のため正常）

証明書警告が出る場合は rootCA が「信頼されたルート証明機関」にあるか（図\$6.5）を確認。whoami.aspが500等になる場合は、古典ASP（図\$11.1のIIS-ASP）が有効か、拡張子がaspかを確認。

11.7 フェーズ8への引き継ぎ値

項目	値
SP entityID	https://sp.plm-lab.local/shibboleth

項目	値
	h
保護対象（予定）	サイト全体（フェーズ8でSPが保護し、REMOTE_USERに識別子が入る）
SP 方式	IIS ネイティブモジュール (SAPI 有効化済み)
サイト	Default Web Site 443/HTTPS、sp 証明書(a))

すべて確認できれば、フェーズ7は完了です。次はフェーズ8 Shibboleth SP をIIS に組み込み、サイト全体を保護) です。

12. フェーズ 8 : Shibboleth SP (IIS ネイティブモジュール・サイト全体保護)

目的：ゲスト Windows にShibboleth SP 3 を導入し、IIS と連携させて Default Web Site 全体をSAML 保護する。SP のSAML 署名・暗号鍵 (b) を生成し、SP メタデータを取得できる状態にする。実際の SSO 成立はフェーズ9 (メタデータ相互登録) 以降で完成する。

前提：フェーズ7でIIS ISAPI 有効・443/HTTPS・確認ページ whoami.asp を用意済み。SP はshibd デモとIIS ネイティブモジュールの2つで構成される。

WSL 版との違い :SP は元々 Windows ネイティブなのでこの部分はWSL 版とほぼ同一。IdP メタデータの8443 補正はフェーズ9 で必要なる構成のIdP は最初から8443 直公開のため。詳細はフェーズ9)。REMOTE_USER に載せる識別子は、フェーズ10 でemailAddress 形式 NameID を割り当てる。

12.1 SP インストーラの入手と導入

- 公式サイト <https://shibboleth.net/downloads/service-provider/latest/> のwin64/ から最新のmsi を入手 (バージョンは固定せず最新を使用)。C:\lab\installers に保存。
- MSI を実行し、既定のまま進める。要点：
 - インストール先は既定の C:\opt\shibboleth-sp。
 - 「Configure IIS support」にチェック IIS ネイティブモジュールを自動構成 実機での表記。
 - 完了後、再起動を求められるので再起動する。

配置の注記 (設計上の推奨) : インストール先はデフォルトの C:\opt\shibboleth-sp を推奨。C:\inetpub 配下は SP の秘密鍵・設定が Web 公開領域に漏洩リスクがあるため不可。集める場合は空を含む。x スキットは shibboleth2.xml 内の ス・keygen 出力先・ログ鍵の スを一斉読替える必要がある。学習・検証ではデフォルトが最も安全

12.2 インストールの確認

- ・ サービス: 「サービス」管理コンソールで **Shibboleth Daemon (Default)** が「実行中／自動／Local System」であること（内部のサービス名は shibd_Default）。
- ・ IIS SP3 のネイティブモジュール方式）: IIS マネージャーでサーバーを選択し「モジュール」に **ShibNative / ShibNative32** C:\opt\shibboleth-sp\lib64\shibboleth\iis7_shib.dll 等、Native/Local）があること。※ SP3 はネイティブモジュール方式のため、旧 ISAPI 方式の *.sso ハンドラーマッピングは表示されないのが正常。下のステータス確認が通れば実質確認済み。
- ・ ステータス 必ず **localhost** で、大文字小文字を区別）: ブラウザで <https://localhost/Shibboleth.sso/Status> を開き、末尾に <Status><OK/></Status> が返ること。

動かない場合は shibd の設定チェック: 管理者コマンドプロンプトで shibd.exe -check を実行 (overall configuration is loadable... が設定読み込み可能。shibd.exe のパスは環境により **sbin64** または **sbin** のいずれか: - 64bit :C:\opt\shibboleth-sp\sbin64\shibd.exe - check - 32bit :C:\opt\shibboleth-sp\sbin\shibd.exe -check

どちらにあるかは dir C:\opt\shibboleth-sp\sbin64\shibd.exe / dir C:\opt\shibboleth-sp\sbin\shibd.exe で確認できる。ログは C:\opt\shibboleth-sp\var\log\shibboleth\shibd.log。

12.3 shibboleth2.xml の編集

C:\opt\shibboleth-sp\etc\shibboleth\shibboleth2.xml を編集します（まず **shibboleth2.xml.org** にバックアップ。タイプミスが最大の事故要因なので慎重に。特に sp.example.org の置換漏れに注意）。変更点は次の4か所です。

(1) <ISAPI> ☒ <Site> ☒ IIS サイトID とホスト名の対応。Default Web Site のID は通常 1)

```
<ISAPI normalizeRequest="true" safeHeaderNames="true">  
  <Site id="1" name="sp.plm-lab.local" scheme="https" port="443"/>  
</ISAPI>
```

(2) <RequestMapper> ☒ <Host> (サイト全体を保護)

「サイト全体」の意味: <ISAPI> の <Site> に登録したサイト（ホスト名 + スキーム + ポート）の、すべてのパスを保護する、という意味（特定パスのみ保護する <Path> 指定との対比で「全体」）。保護される／されないは、<Site> に登録したかどうかで決まる。登録していないポート（例: HTTP の 80/9012/9013）には SP は関与せず、従来の認証（Cookie 等）のまま動作する。HTTPS だから保護される・HTTP だから保護されない」のではなく、<Site> に書いたかどうかで判断基準。この性質が、\$17 の並行運用（HTTP → 従来認証 HTTPS → SSO）の根拠になる。

```
<RequestMapper type="Native">  
  <RequestMap>  
    <Host name="sp.plm-lab.local" authType="shibboleth" requireSession="true"/>
```

```
</RequestMap>
</RequestMapper>
```

requireSession="true" により、このホストへの全アクセスがセッション必須 (= 未認証なら IdP へ) になります。これが「サイト全体保護」の設定です。

(3) <ApplicationDefaults> 内の entityID と REMOTE_USER

```
<ApplicationDefaults entityID="https://sp.plm-lab.local/shibboleth"
  REMOTE_USER="eppn persistent-id targeted-id">
```

REMOTE_USER は「先頭から最初に値のある属性」が採用されます。識別子 (emailAddress 形式 NameID) をここに載せる最終設定はフェーズ 10で行います (IdP 側の NameID/属性の出し方と対で決めるため)。本フェーズでは既定のままにしておきます。

(4) <SSO> に IdP 内の entityID を指定

```
<SSO entityID="https://idp.plm-lab.local/idp/shibboleth">
  SAML2
</SSO>
```

IdP のメタデータ (<MetadataProvider>) の登録はフェーズ 9で行います。本フェーズでは entityID の指定までにとどめます。<MetadataProvider> を追加する際は、<Sessions> の外、<Errors> の後ろに置く (<Sessions> 内に置くとスキーマ違反で shibd が起動しない。フェーズ 9で詳述)。

12.4 SP 鍵 (b) の生成

SP の SAML 署名・暗号化証明書 (b) を、正しいホスト名・entityID で生成します。管理者コマンドプロンプトで：

```
cd C:\opt\shibboleth-sp\etc\shibboleth
keygen.bat -h sp.plm-lab.local -e https://sp.plm-lab.local/shibboleth -y 10
```

sp-signing-cert.pem / sp-encrypt-cert.pem 等が生成されます。これはフェーズ 2の(a)とは別物で、メタデータ交換で IdP に渡す (b) 内の CA 信頼・ホスト名一致は不要。MSI が既定鍵を生成済みの場合もありますが、entityID/ホスト名を正しくするため上記で作り直します。

12.5 反映 (IIS 完全再起動)

<ISAPI> を変更したときは IIS の完全再起動が必要です。

```
Restart-Service shibd_Default
iisreset
```

12.6 動作確認

#	確認内容	期待結果
1	shibd 稼働	「Shibboleth Daemon (Default)」が実行中
2	設定読み込み	shibd.exe -check (sbin64 または sbin) が overall configuration is loadable
3	Status	https://localhost/Shibboleth.sso/Status が <Status><OK/></Status>
4	SP メタデータ	https://sp.plm-lab.local/Shibboleth.sso/Metadata から SP メタデータ (XML) を返す。中に sp.example.org が残っていない (すべて sp.plm-lab.local)
5	保護の発火	https://sp.plm-lab.local/whoami.asp にアクセスすると SP がセッションを要求する (この時点では IdP メタデータ未登録のため No MetadataProvider available. 等になるのは想定どおり。requireSession が効いている証拠)

確認の SP メタデータ (Shibboleth.sso/Metadata) は、フェーズ 9 で IdP に登録する S P メタデータとして使います。ファイルに保存しておくと便利です。

12.7 フェーズ 9 への引き継ぎ値

項目	値
SP entityID	https://sp.plm-lab.local/shibboleth
SP メタデータ URL	https://sp.plm-lab.local/Shibboleth.sso/Metadata
SP の ACS (想定)	https://sp.plm-lab.local/Shibboleth.sso/SAML2/POST
SP 署名・暗号鍵 (b)	C:\opt\shibboleth-sp\etc\shibboleth\sp-*-cert.pem
次工程	フェーズ 9 :IdP メタデータを SP に登録し SP メタデータを IdP に登録。本構成では IdP が 8443 直公開のため :8443 補正は不要

すべて確認できれば、フェーズ 8 は完了です。次はフェーズ 9 (メタデータ交換・初回 SSO 成立) です。

13. フェーズ 9 : メタデータ交換 (IdP ↔ SP の相互信頼 ・ 初回 SSO 成立)

目的 : IdP と SP のメタデータを静的 (ファイル) に相互登録し、両者が信頼し合って SAML の往復が成立する状態にする。フェーズ 8 で出た No MetadataProvider available. を解消し、<https://sp.plm-lab.local/whoami.asp> への承認アクセスから IdP のログイン画面 (8443) へ遷移 - 識別子 + パスワードでログイン - SP に戻ってセッション確立、という一連を確認する。

本フェーズのゴール : 「SSO の往復が成立し、SP セッションが張れて保護ページに到達できる」ここまで REMOTE_USER に識別子を載せる最終設定はフェーズ 10。本フェーズ完了時点では REMOTE_USER は空でも合格。

WSL 版との違い (8443 補正 ・ 実機結果) : \\wsl\$ /mnt/c の受け渡しや chown は不要で、すべて同一 Windows 上のファイルコピーで済む。ただし install.bat はホスト名ベースでメタデータを生成するため、エンドポイントの Location がポートなし (=443) で出力されることがある (実機でもポートなしだった)。その場合は WSL 版と同様に 8443 へ補正が必要 (補正の手段が sed → PowerShell に変わるだけ)。13.1 で実際の Location を確認し、ポートなしなら補正する。

13.1 IdP メタデータのエンドポイント確認と 8443 補正

IdP のメタデータは C:\opt\shibboleth-idp\metadata\idp-metadata.xml にあります。まず、各エンドポイントの Location が 8443 付きかポートなし (=443) を確認します (管理者 PowerShell)。

```
Select-String -Path "C:\opt\shibboleth-idp\metadata\idp-metadata.xml" -Pattern 'Location = "[^"]*"' |  
ForEach-Object { $_.Matches.Value } | Sort-Object -Unique
```

実機ではポートなし ([https://idp.plm-lab.local/idp/...](https://idp.plm-lab.local/idp/)) で生成された。このままを SP はブラウザを 443 回 IIS/SP 側へ送ってしまい SSO が壊れるため SP 側コピーしたら 8443 へ補正する (entityID 行はポートなしのまま残す)。

1) IdP メタデータを SP 側へコピー

```
Copy-Item "C:\opt\shibboleth-idp\metadata\idp-metadata.xml" `"  
"C:\opt\shibboleth-sp\etc\shibboleth\idp-metadata.xml" -Force
```

2) コピー先のエンドポイントを 8443 に補正 (entityID は元に戻す)

```
$f = "C:\opt\shibboleth-sp\etc\shibboleth\idp-metadata.xml"  
(Get-Content $f -Raw) `"  
-replace 'https://idp\.plm-lab\.local/idp/', 'https://idp.plm-lab.local:8443/idp/' `"  
-replace 'entityID="https://idp\.plm-lab\.local:8443/idp/shibboleth"', 'entityID="https://idp.plm-lab.local/idp/shibboleth"' |  
Set-Content $f -Encoding UTF8
```

#3) 確認 : Location が :8443、entityID はポートなし

```
Select-String -Path $f -Pattern 'Location="[^\"]*"' | ForEach-Object { $_.Matches.Value } | Sort-Object -Unique
```

```
Select-String -Path $f -Pattern 'entityID="[^\"]*"' | ForEach-Object { $_.Matches.Value } | Sort-Object -Unique
```

期待 : Location がすべて https://idp.plm-lab.local:8443/idp/...、entityID は https://idp.plm-lab.local/idp/shibboleth 形式でポートなし。識別子であってアクセス先 URL ではないためポートを付さない。もし最初から Location が :8443 付きで生成されていた場合は手順のコピーのみで補正は不要。

13.2 SP に IdP メタデータを登録

C:\opt\shibboleth-sp\etc\shibboleth\shibboleth2.xml に IdP メタデータの <MetadataProvider> を追加します。配置場所が重要で <MetadataProvider> は <Sessions> の中ではなく、</Sessions> の後・<Errors .../> の下に置きます (Sessions 内 置くとスキーマ違反 element 'MetadataProvider' is not allowed for content model ... で shibd が起動しません)。既定ファイルの「Example of locally maintained metadata」コメントの置き場がここです。

```
</Sessions>
```

```
<Errors supportContact="root@localhost"
  helpLocation="/about.html"
  styleSheet="/shibboleth-sp/main.css"/>
```

<!-- ここ (Sessions の外・Errors の下) に追加 -->

```
<MetadataProvider type="XML" validate="true" path="idp-metadata.xml"/>
```

保存後設定チェックしてから SP を再起動します (shibd.exe は sb64 または sbin)。

```
C:\opt\shibboleth-sp\sbin64\shibd.exe -check # 無ければ sb64\shibd.exe -check
```

```
Restart-Service shibd_Default
```

```
iisreset
```

これで No MetadataProvider available. は解消します。https://localhost/Shibboleth.h.sso/Status が引き続き <OK/> であることも確認

13.3 SP メタデータの取得と IdP への配置

SP のメタデータを取得して IdP 側に配置します。ブラウザで https://sp.plm-lab.local/Shibboleth.sso/Metadata を開き、XML を **sp-metadata.xml** として保存 (フェーズ 8 で保存済みならそれを使用)。これを IdP のメタデータ領域へコピーします (同一 Windows なので単純コピー)。

```
Copy-Item "C:\Users\<ユーザー>\Downloads\sp-metadata.xml" `
"C:\opt\shibboleth-idp\metadata\sp-metadata.xml" -Force
```

WSL 版のような chown tomcat は不要 (Windows のサービスは Local System で動作し、ファイル所有権の付け替えは不要)。

13.4 IdP に SP メタデータを登録

C:\opt\shibboleth-idp\conf\metadata-providers.xml を編集し、既定の <MetadataProvider id="ShibbolethMetadata" xsi:type="ChainingMetadataProvider"> と、それを閉じる </MetadataProvider> の間 (チェーンの内側) に SP メタデータの FilesystemMetadataProvider を追加します (コメントアウトされた LocalMetadata 見本の位置が最適)。

```
<MetadataProvider id="LocalSP" xsi:type="FilesystemMetadataProvider"
    metadataFile="%{idp.home}/metadata/sp-metadata.xml"/>
```

IdP に設定を反映させます。学習環境では Tomcat 再起動が確実です (reload-service はアクセス制御で弾かれることがある)。

```
Restart-Service Tomcat10
```

```
Start-Sleep -Seconds 20
```

```
# SP メタデータ (entityID: https://sp.plm-lab.local/shibboleth) の読み込み・エラー無しを確認
```

```
Select-String -Path "C:\opt\shibboleth-idp\logs\idp-process.log" -Pattern 'LocalSP|sp.plm-lab.local' | Select-Object -Last 10
```

```
# 期待: "FilesystemMetadataResolver LocalSP: New metadata successfully loaded ..." が出る
```

13.5 時刻の確認

SAML はクロックスキューに敏感ですが、本構成は IdP も SP も同一の Windows 図のため時刻は常に一致し、WSL 版のような OS 間のずれは発生しません。Get-Date が妥当な現在時刻であることを確認する程度で十分です。

13.6 初回 SSO の確認

ゲスト Windows のブラウザ 新規プライベートウィンドウ推奨で:

1. <https://sp.plm-lab.local/whoami.asp> を開く
2. IdP のログイン画面 (<https://idp.plm-lab.local:8443/idp/...>) に遷移する
3. ユーザー名 01PLM01、パスワード 01PLM01 (フェーズ 3 の Joe アカウント) でログイン
4. SP に戻り、whoami.asp が表示される (保護ページ到達)

#	確認内容	期待結果
1	保護ページ到達	承認アクセスで IdP ログイン画面 (8443) へ遷移
2	認証	uid=01PLM01 でログインできる (LDAP 認証)

#	確認内容	期待結果
		成立)
3	復路	SP に戻り whoami.asp が開ける (セッション確立)
4	セッション	https://sp.plm-lab.local/Shibboleth.s so/Session に有効なセッションがある local host では不可。ログインしたホス ト名で開く)

この時点で REMOTE_USER は空でも合格。識別子 (emailAddress 形式 NameID) を R
EMOTE_USER に載せるのはフェーズ 10。

13.7 つまずいたときの切り分け

- **IdP ログイン画面に飛ばず 443 に行ってしまう** → 13.1 のエンドポイントが 8443 になっている
か。SP 側 idp-metadata.xml の Location を確認。
- **shibd が起動しない / element 'MetadataProvider' is not allowed ...** → <Metadata
Provider> を <Sessions> 内 置 る </Sessions> の後、<Errors> の下へ移動 (13.2)。
- **IdP 側で「SAML2 SSO profile is not configured for relying party ...sp...」** → IdP に
SP メタデータを読み込めない (13.4)。idp-process.log と metadata-providers.xml の SP メタ
データの entityID を確認
- ログイン時に **Pool is empty and connection creation failed (PoolExhaustedExc
eption)** → IdP が LDAP に接続・bind できていない。ApacheDS が起動していて **10389** が **LI
STEN** していても、エントリもパスワードも正しくても発生することがある。
切り分けと対処：
 1. 最有力：**conf / credentials** 配下に余計な **.properties** (バックアップコ
ピー) が無いか。Windows のコピーで作れる **ldap - Copy.properties** は拡張子が pro
perties のままなので IdP に読み込まれ、編集前の既定値 (**uid=myservice,ou=
system / dc=example,dc=org / myServicePassword**) が使われてしまう。
起動時の **WARN ... Duplicate properties were detected** かその兆候 → 拡張子を変える (
dap.properties.orig) か IdP の外へ退避し Tomcat 再起動 (§9.4)。
 2. ldap.properties の trustCertificates / trustStore が有効になっていないか (平
文 LDAP では不要 コメントアウトする §9.4)。
 3. 真の原因を見るには LDAP のデバッグログを一時的に有効化：conf\logba
ck.xml に <logger name="org.ldaptive" level="DEBUG"/> を追加 → Tomcat 再起
動 → ログイン試行 → ログの **bindDn= / baseDn=** を確認。設定した値と違
う値が出ていれば 1. のパターン。
 4. 参考：この事象は **hosts** の実 IP 化とは無関係。IdP-LDAP は ldap://localhos
t:10389 のレイプバック接続 ログ出る 192.168.x.x (はブラウザの IP)。
- **署名/復号エラー** → メタデータ内の (b) 証明書と実鍵の不一致。SP/IdP のメタデータ最新か
keygen 後に再取得したか) を確認。

- ログ:SP はC:\opt\shibboleth-sp\var\log\shibboleth\shibd.log、IdP はC:\opt\shibboleth-idp\logs\idp-process.log。

すべて確認できれば、フェーズ9は完了です（初回 SSO 成立）。次はフェーズ 10（emailAddress 形式 NameID をREMOTE_USER に載せる）です。

14. フェーズ 10：属性連携（emailAddress 形式 NameID を REMOTE_USER に載せる）

目的：認証されたユーザーの mail（例 01PLM01@plm-lab.local）を IdP から emailAddress 形式の NameID として SP へ渡し、SP 側で REMOTE_USER にマッピングする。最終的に whoami.asp の REMOTE_USER に 01PLM01@plm-lab.local が表示される状態にする（顧客 Entra の形式に準拠）。

WSL 版との違い：WSL 版は「uid を unspecified 形式の NameID」だったが、本構成は「mail を emailAddress 形式の NameID」。顧客の本番（Entra ID）が emailAddress 形式の NameID を送付している。PLM 側で @ の前を切り出して従来の識別番号に変換する処理は PLM アプリの責務であり、本フェーズの対象外（SP は 01PLM01@plm-lab.local をそのまま REMOTE_USER に載せるところまで）。設定は最も細かいので 1 か所ずつ変更して確認する。

14.1 IdP：mail 属性の確認（attribute-resolver.xml：ドメインを変更）

IdP 5 の既定 attribute-resolver.xml にはすでに mail の Template 定義が入っている（uid にドメインを付けてメール形式を作る）。既定ドメインが @example.org になっているため、これを @plm-lab.local に変更するだけでよい（LDAP DataConnector の新設は不要）。

```
<!-- 既存の mail 定義。Template のドメインだけ変更する -->
<AttributeDefinition id="mail" xsi:type="Template">
  <InputAttributeDefinition ref="uid" />
  <Template><![CDATA[${uid}@plm-lab.local}]]></Template> <!-- @example.org から変更 -->
</AttributeDefinition>
```

これで mail は 01PLM01@plm-lab.local になる（ログイン名 uid = 個人番号にドメインを付す形。顧客の「個人番号@ドメイン」の考え方にも合致）。

⚠ 二重定義に注意：既定定義を変更するので、新しい <AttributeDefinition id="mail" ...> を別途追加しないこと。id="mail" が 2 つあると IdP 起動時に WARN ... Duplicate Definition 'mail' ...（重複定義）となる。1 つだけにする。

別解（LDAP の mail を直接引く）：LDAP の mail 属性をそのまま使いたい場合は LDAP DataConnector を加えて mail を取得する方法もあるが、手数が増える。本書は既定テンプレートのドメイン変更（上記）を採る。

14.2 IdP : mail を対象 SP へ解放 (attribute-filter.xml)

C:\opt\shibboleth-idp\conf\attribute-filter.xml に SP(sp.plm-lab.local) へ mail を解放するポリシーを追加します。

```
<AttributeFilterPolicy id="releaseMailToPlmSP">
  <PolicyRequirementRule xsi:type="Requester"
    value="https://sp.plm-lab.local/shibboleth"/>
  <AttributeRule attributeID="mail">
    <PermitValueRule xsi:type="ANY"/>
  </AttributeRule>
</AttributeFilterPolicy>
```

14.3 IdP : mail を emailAddress 形式 NameID として生成 (saml-nameid.xml)

フェーズ 5 §9.5 で shibboleth.SAML2NameIDGenerators に次の生成器を追加済みのはず
です (未追加ならここで追加。重複させない)。

```
<bean parent="shibboleth.SAML2AttributeSourcedGenerator"
  p:omitQualifiers="true"
  p:format="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress"
  p:attributeSourceIds="#{'mail'}" />
```

14.4 IdP : 対象 SP に emailAddress 形式を優先させる (relying-party.xml)

C:\opt\shibboleth-idp\conf\relying-party.xml の <util:list id="shibboleth.RelyingPartyOverrides"> に、対象 SP 向けのオーバーライドを追加します。

```
<bean parent="RelyingPartyByName"
  c:relyingPartyIds="https://sp.plm-lab.local/shibboleth">
  <property name="profileConfigurations">
    <list>
      <bean parent="SAML2.SSO"
        p:nameIDFormatPrecedence="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress"/>
    </list>
  </property>
</bean>
```

設定反映 (Tomcat 再起動) 確認。

Restart-Service Tomcat10

Start-Sleep -Seconds 20

Select-String -Path "C:\opt\shibboleth-idp\logs\idp-process.log" -Pattern 'ERROR' | Select-Object -Last 20

14.5 SP : NameID を REMOTE_USER にマップ

(1) **attribute-map.xml** (C:\opt\shibboleth-sp\etc\shibboleth\attribute-map.xml) に email Address 形式の NameID を属性 mail として取り込むデコーダを追加します。

```
<Attribute name="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress" id="mail">
  <AttributeDecoder xsi:type="NameIDAttributeDecoder" formatter="$Name"/>
</Attribute>
```

(2) **shibboleth2.xml** の <ApplicationDefaults> の REMOTE_USER に、先頭で mail を使うよう変更します (顧客の優先リスト方式に倣う)。

```
<ApplicationDefaults entityID="https://sp.plm-lab.local/shibboleth"
  REMOTE_USER="mail eppn persistent-id targeted-id">
```

反映 &P 再起動 shibd.exe (おはbin64 おはbin)。

```
C:\opt\shibboleth-sp\sbin64\shibd.exe -check # overall configuration is loadable
Restart-Service shibd_Default
iisreset
```

partner-metadata.xml に注意 : 既定の shibboleth2.xml には <MetadataProvider type="XML" validate="true" path="partner-metadata.xml"/> の行が含まれることがある。このファイルが存在しない場合は validate="true" によりエラー要因になり得るため、該当行をコメントアウト (または削除) しておく (フェーズで追加した idp-metadata.xml の行を有効にする)。

ログ確認の注意 (時刻でフィルタ) : idp-process.log は追記のため Select-String は過去の行 (修正前の WARN など) も拾う。再起動後の状態を見るときは、最新の Shibboleth IdP Version 5.2.3 行以降、または時刻でフィルタして確認する (例:... | Where-Object { \$_.Line -match '2026-07-11 11:5' })。

14.6 動作確認 (最終ゴール)

ゲスト Windows のブラウザで新しいプライベートウィンドウで:

1. https://sp.plm-lab.local/whoami.asp を開く
2. IdP ログイン画面 (8443) - 01PLM01 / 01PLM01 でログイン
3. whoami.asp に戻し **REMOTE_USER = [01PLM01@plm-lab.local]** と表示される

#	確認内容	期待結果
1	SSO 往復	ログイン後 whoami.asp に戻る
2	識別子の受け渡し	REMOTE_USER = [01PLM01@plm-lab.local] (メール形式が入る)
3	セッション内容	https://sp.plm-lab.local/Shibboleth.sso/Session の Attributes に mail 属性 Name

#	確認内容	期待結果
		ID) が見える

これが表示できれば、本手順書の目標 (**emailAddress** 形式の識別子による **SSO** 連携) は達成です。

14.7 つまづいたときの切り分け

- **REMOTE_USER** が空のまま → ① IdP の **NameID** を出しているか (Shibboleth.sso/Session の Attributes/NameID を確認)、② SP の attribute-map.xml の形式 (emailAddress) と id ⓧmail) が一致しているか、③ REMOTE_USER の先頭に mail があるか。
- **NameID** が **transient** のまま → 14.4 の nameIDFormatPrecedence ⓧemailAddress) が対象 SP に効いているか、relying-party.xml の entityID を確認。
- **mail** が解決できない (**IdP** エラー) → 14.1 の LDAP DataConnector と ldap.properties の resolver 設定 LDAP の mail 属性 (フェーズ 3 で投入済み) を確認。idp-process.log を確認
- 確認ツール : ブラウザ拡張「SAML-tracer」で IdP→SP の SAML Response 内の <NameID> に 01PLM01@plm-lab.local が入っているかを直接確認できる。
- ログ: SP=C:\opt\shibboleth-sp\var\log\shibboleth\shibd.log、IdP=C:\opt\shibboleth-idp\logs\idp-process.log。

14.8 実運用 (PLM) への接続に向けた補足

アプリから見た認証情報の受け取り方 (3 層の役割分担)

Web アプリ (whoami.asp や実際の PLM) が認証済ユーザーを知りたく理解して、アプリは **SAML** アサーションも **Cookie** も直接は見ない。**SP** が確定した **REMOTE_USER** を読むだけである。

【層 1】SAML アサーション (IdP → SP)

IdP が「この人は 01PLM01@plm-lab.local」と署名付きで SP に渡す

※ ログイン時に 1 回だけ (ブラウザ経由で SP の ACS へ POST)

↓ SP (shibd) が署名・有効期限・発行者を検証し、サーバ側セッションに保存

【層 2】SP のセッション (サーバ側) + Cookie (引換券)

認証情報の実体 (NameID・属性・認証時刻・IdP) は shibd がサーバ側に保持

ブラウザには「セッション ID だけ」の暗号化 Cookie (_shibsession_...) を渡す

↓ リクエストのたびに SP が Cookie からサーバ側セッションを引き当て

【層 3】REMOTE_USER (IIS のサーバ変数) ← ★ アプリが見るのはここだけ

SP が「このリクエストは 01PLM01@plm-lab.local」と確定して IIS に渡す

↓

whoami.asp / PLM アプリが Request.ServerVariables("REMOTE_USER") で読む

要点 :

- **SAML** アサーションはアプリに渡らない。検証するのはSP(Shibboleth)。アサーションが飛ぶのはログイン時の1回だけで、以後の毎リクエストで送られるわけではない。アプリが **SAML** を解析する必要はない。
- **Cookie** からユーザー情報は読めない。_shibsession_... は暗号化された引換券 (セッション ID) にすぎず、user=01PLM01 のような情報は入っていない。復号できるのはSP だけ (アプリ読めず) ユーザーが Cookie を書き換えてしまえば意味ない。
- **Cookie** が指しているのは「どのサーバ側セッションか」。「認証済みか」+「誰か」の情報はサーバ側セッション ID。Cookie はそれを指すポインタ。その中身は /Shibboleth.sso/Session で確認できる (§15.8(4))。
- アプリは **REMOTE_USER** を読むだけでよい。SAML の検証・Cookie の復号・IdP との通信はすべてSP が肩代わりする。これをSP が前段で置く最大の利点。

アプリ側の実装イメージ：

```
<%-- 古典 ASP (本書の whoami.asp) --%>
```

```
<%= Request.ServerVariables("REMOTE_USER") %>
```

//ASP.NET (実際の PLM)

```
string user = Request.ServerVariables["REMOTE_USER"]; // "01PLM01@plm-lab.local"
```

```
string id = user.Split('@')[0]; // "01PLM01" ← @ の前を切り出す
```

//以降、id (個人番号) で自 DB の認可判定を行う

REMOTE_USER 以外の属性を渡したい場合 :attribute-map.xml でマップした属性は、SP がサーバ変数 (**HTTP** ヘッダー) としてアプリに渡すことができる (safe HeaderNames の設定により名前変換)。顧客の shibboleth2.xml には **REMOTE_USER**="azure_id azure_userid gid_id" や cflag / pcode / ccode といった属性も、この仕組みでアプリに渡す想定。本書検証では **REMOTE_USER** 1つで足りるため、追加属性の受け渡しは実装していない。

PLM 側の責務 (再掲)

本フェーズで **REMOTE_USER** にメール形式の識別子 (01PLM01@plm-lab.local) が入るようになった。実際の PLM ではこの **REMOTE_USER** (IIS のサーバ変数) を PLM 側が読み取り @ の前 (**01PLM01**) を切り出して従来の識別番号として自 **DB** で認可判定する。この切り出し・変換は PLM アプリ (Web.config の認証方式スイッチ + 実装) の責務であり、SP/IdP の構築範囲外。SP はサイト全枠保護するため PLM の各ページはすべて認証必須となる。顧客本番では IdP が Entra ID に替わりますが SP 側 (attribute-map / **REMOTE_USER**) の考え方は同じ。

15. フェーズ 11 : 結合テスト (SSO の通し確認・ログ・再現性)

目的 : 構築した SSO (Windows 版) の学習環境の構築を完結する。

15.1 ログインの通し確認 (クリーンな状態から)

ゲスト Windows のブラウザで新しいプライベートウィンドウで:

1. <https://sp.plm-lab.local/whoami.asp> を開く - IdP ログイン画面 (8443) へ遷移
2. 01PLM01 / 01PLM01 でログイン
3. whoami.asp に戻し **REMOTE_USER = [01PLM01@plm-lab.local]** が表示される

15.2 別ユーザーでの再現性

値が固定でなく、認証したユーザーの識別子が反映されることを確認します。別のプライベートウィンドウで **01PLM02 / 01PLM02** (フェーズ3で作成した2人目) でログインし、REMOTE_USER = [01PLM02@plm-lab.local] になることを確認します。

15.3 セッションの確認

- [https://sp.plm-lab.local/Shibboleth.sso/Session ...](https://sp.plm-lab.local/Shibboleth.sso/Session) 現在のセッションのNameID・属性・IdP entityID などが確認できる。必ずログインしたホスト名 (**sp.plm-lab.local**) で開くこと。 <https://localhost/...> で開くとセッションCookieはsp.plm-lab.localに紐づいてるため送れずA valid session was not found. になる (セッションが無のではなく、ホスト名違でCookieが届かない)。Attributes 欄にmailⓧemailAddress 形式(NameID) が見える
- [https://localhost/Shibboleth.sso/Status ...](https://localhost/Shibboleth.sso/Status) SP の稼働状態 (<OK/>)。Status はlocalhost で可。Status とSession でアクセスするホスト名異なる点に注意

15.4 ログアウト (本手順の学習範囲では対象外)

本手順の学習目的は「SSOの成立」であり、ログアウト (特に **SLO**: シングルログアウト) は対象外とする。終了する場合はブラウザ (プライベートウィンドウ) を閉じることで足りる。

補足 (仕組み) : <Logout>SAML2 Local</Logout> 既定はSAML2 SLOを試みるが、IdP 5 は既定でSLO プロファイルが整備されていないため /Shibboleth.sso/Logout はWeb Login Service - Error: NoHandlerFoundExceptionⓧIdP 由来 になる。SP ローカルのみで完結させたい場合は<Logout>Local</Logout> に変更するとエラーなくSPセッションを破棄できるが、IdP 側セッションは残るため直後の再アクセスはパスワードなしで再ログインされる (SSO 本来の挙動)。完全に切るにはブラウザを閉じる。完全な SLO はIdP 側の追加設定が必要で本手順の範囲外。

15.5 ログの読み方

IdP (Windows) : C:\opt\shibboleth-idp\logs\ - idp-process.log ... 一般的な処理ログ。エラー調査の起点。追記式のため、再起動後の確認は最新の **Shibboleth IdP Version** 行以降、または時刻でフィルタして見る (古い WARN を拾わないため)。 - idp-audit.log ... 監査ログ。1行1認証で、いつ・どの利用者が・どのSPへ・何を出したかが分かる。

「どのログインか」を追うのに最適 - idp-warn.log ... 警告のみ抽出。

SP (Windows) :C:\opt\shibboleth-sp\var\log\shibboleth\ - shibd.log ... SP デーモンのログ。アサーション受領・セッション確立・エラーはここ。 - transaction.log ... セッション単位の記録。

成功時の典型 : IdP の idp-audit.log に 01PLM01 の認証が出て SP の shibd.log に「new session created」相当の行が出る。うまくいかないときは、まず IdP 監査ログで認証解放を確認するかを見て IdP 側か SP 側かを切り分ける。

15.6 再起動後の堅牢性 (Windows 版は自動起動で堅牢)

純 **Windows** 版の利点 : IdP (Tomcat) も SP (IIS/shibd) も LDAP (ApacheDS) もすべて **Windows** サービスとして動く。WSL 版のような「オンデマンド起動 (ターミナルを開くまで IdP が起動しない)」問題は発生しない。各サービスが自動起動になっていれば、Windows を再起動するだけで SSO が復旧する。

各サービスの自動起動を確認 :

```
Get-Service Tomcat10, shibd_Default, W3SVC |  
Select-Object Name, Status, StartType  
# ApacheDS (表示名で取得)  
Get-Service | Where-Object { $_.DisplayName -like "*ApacheDS*" } |  
Select-Object Name, Status, StartType
```

- いずれも **StartType** が **Automatic**、Status が **Running** であること (W3SVC は IIS、Tomcat10 は IdP、shibd_Default は SP、ApacheDS は LDAP)。
- Automatic でないサービスがあれば設定 : Set-Service <名前> -StartupType Automatic。
- サービスの起動順の依存 : IdP は起動時 LDAP (ApacheDS) へ接続するが、各サービスは独立起動のため、まれに ApacheDS より先に Tomcat が上がっても、認証は要求時に行われるため実害は出にくい。もし再起動直後の初回ログインが失敗する場合は、少し待つか ApacheDS - Tomcat の順に手動再起動する。

再起動テスト : ゲスト Windows を再起動 → ログオン後 (各サービスが自動起動) → そのまま <https://sp.plm-lab.local/whoami.asp> で SSO が通ることを確認する (ターミナルを開くなどの操作は不要)。

15.7 よくある問題の早見表 (総まとめ・Windows 版)

症状	主な原因	対処 (参照)
IdP ログイン後 403 に飛ぶ / SSO が壊れる	IdP メタデータのエンドポイント未修正	SP 側 dp-metadata.xml を 8443 に修正 (§13.1)
shibd 起動失敗 (content model エラー)	<MetadataProvider> を Sessions 内配置	</Sessions> の後・<Errors> の下へ (§13.2)
shibd 起動失敗 (metadata 読みエラー)	存在しない partner-metadata.xml	該当 MetadataProvider をコメントアウト (§14.5)

症状	主な原因	対処 (参照)
/idp/profile/status が ServletException	JSTL 未導入	JSTL 2 jar を追加し build.bat §9.3)
IdP 起動時 Duplicate Definition 'mail'	attribute-resolver に mail 定義が重なる	mail 定義を1つに既存のドメイン変更のみ §14.1)
REMOTE_USER が空	NameID 未生成/SP マッピング不足	§14.2~14.5、Session/SAML-tracer で切り分ける
証明書警告	rootCA 未信頼	信頼リストに登録 §6.5)
Shibboleth.sso/Session から Valid session...	localhost で開いている	ログインホスト名 sp.plm-lab.local で開く §15.3)
LDAP に繋がらない	ポート違い (既定10389)	ldap://localhost:10389 §7・§9.4)
wwwroot に書き込めない	ブレイク Administrator 承認モード	昇格プロセス作成 §11.5)

15.8 テスト実施時の留意点と確認手段 (証明書警告・プライベートウィンドウ・セッション)

構築後、社内のテスト環境で SSO を検証・運用する際に、仕組みを理解したうえで判断すべき点をまとめる。

(1) 証明書警告を許容してよいか (クライアントへのルート証明書配布を省略する場合)

警告が出る理由 : ①ブラウザは HTTPS 接続の署名か ②ホスト名 (SAN) が一致するか ③有効期限内か」を検証する。本環境の証明書は自前の内部 CA (PLM-Lab Root CA) が署名しており、クライアントがこの CA を信頼していないため ①が失敗して警告になる (②③は満たしている)。

警告を無視して進むことの意味 :

- 失われるもの = サーバの真正性の保証 : 「接続先が本当に自分たちのサーバか」をブラウザが保証できない。理屈上は中間者攻撃を検知できない。
- 失われないもの = 通信の暗号化 : TLS の暗号化自体は有効で、パスワードや Cookie が平文で流れるわけではない (HTTP との決定的違い)。暗号化はされているが、相手先が本物である保証はない状態

社内テスト環境での判断 : クローズドな社内ネットワークで、テストユーザー (01PLM01 等) による検証を行う限り、証明書配布を省略し警告を許容する運用は現実的に

許容できる（中間者攻撃は通信経路に割り込める場合に顕在化するもので、社内テストでのリスクは実務上小さい）。ただし次の2点に留意：

- 警告は **SP** と **IdP** の両方で出る。plmdev.plm-lab.local と idp.plm-lab.local は別ホスト名のため。SAML の往復で2回通過する必要があるため、テスト手順に明記しておく。
- 本番では必ず正規の証明書を使う（社内 CA/AD CS 等。\$16.6）。テストでの割り切りを本番に持ち込まないよう線引きを明示する。

(2) プライベートウィンドウを使う理由と、使わない場合のリスク

理由：**SSO** は認証状態が **Cookie** として残る仕組みだから。プライベートウィンドウは毎回クリーンな（未認証の）状態からテストを開始するための「状態リセットの道具」である。残るのは次の2種類：

- SP** のセッション **Cookie** (shibsession_...)：「このユーザーは認証済み」という SP 側の状態
- IdP** のセッション **Cookie** (shib_idp_session)：「このユーザーは IdP でログイン済み」という状態が残っていると IdP はパスワードを聞かずにアサーションを再発行する（SSO 本来の挙動）。

使わない場合のリスク（テストでの誤判定）：

状況	起こること	リスク
前回のセッションが残っている	ログイン画面が出ずページが表示される	「SSO が動いた」と誤認するが、実は前回のセッションで入れただけ。設定変更の効果を検証できていない
別ユーザーでテストしたい	前ユーザーのセッションが残るため IdP でログインできない	再現性テスト（\$15.2）が実施できない
設定変更後の確認	変更前のセッションが生きている	修正の効果を誤判定する

使い分け：認証フローそのものを確認するテスト（ログインできる/別ユーザーで入れる/設定変更の効果）はプライベートウィンドウ必須。逆に **SSO** の利点（1回のログインで複数環境に入れる）を確認するときはあえて同じウィンドウを使う（図3）。

(3) 複数環境間での認証スキップ（SSO の実体）

\$17 の並行運用のように、同一ホスト名の別ポートで複数環境（例：① https://plmdev.plm-lab.local:9112/② https://plmdev.plm-lab.local:9113/）を SSO 保護した場合 ① でログインすれば、② へのアクセス時には認証がスキップされる。認証情報はサーバ間で直接受け渡されるのではなく、ブラウザが **Cookie** を運ぶことで引き継がれる。メカニズムは2段階：

- メカニズム 1：**SP** のセッション **Cookie** が共有される（本構成の場合）方式（同一 entityID）+ 同一ホスト名の別ポートでは **Cookie** はポートを区別しないため shibsession_... の値が SP は「セッションあり」と判断し IdP に行くことなく即座に許可（SAML の往復すら発生しない）。

- メカニズム 2 : IdP のセッション Cookie (SSO の本質) 仮に SP セッションが共有可能な構成 (別ホスト名の SP、方式 2 の別 EntityID 等) でも、②にアクセスすると SP は IdP へリダイレクトする。このときブラウザが IdP のセッション Cookie (shib_idp_session) を送るため IdP はログイン画面を出さずにアサーションを再発行する。ユーザーからはパスワードを聞かずにログインできる。これが「1 回ログインすれば他システムにも入る」という SSO の本質。

① でログイン

└→ SP が Cookie 発行 (plmdev.plm-lab.local 宛) ← メカニズム 1

└→ IdP が Cookie 発行 (idp.plm-lab.local 宛) ← メカニズム 2

② にアクセス

└→ ブラウザが SP Cookie を送信 → SP が「認証済み」と判断 → 即許可 (本構成はここで完結)

└→ (SP Cookie が無い構成なら) IdP へリダイレクト → ブラウザが IdP Cookie を送信 → IdP が「認証済み」と判断 → パスワード不要でアサーション発行 → SP へ戻る

テスト時の落とし穴 : 「②で認証が求められない」のを見て「②の保護が効いていない」と誤解しやすい。実際は①のセッションが生きているだけ。②単体の保護を検証するなら、プライベートウィンドウを開き直して (クリーンな状態で) ②に直接アクセスし、ログイン画面が出ることを確認する (②と繋がる)。

(4) Cookie ・ 認証情報の確認手段

重要 : Cookie の中身 (認証情報) は読めない。SP/IdP のセッション Cookie は暗号化された不透明な値であり、user=01PLM01 のような平文は入っていない。中身は「サーバ側セッションを指す引換券 (セッション ID) 」にすぎず、実際の認証情報 (NameID ・ 属性 ・ 認証時刻) は shibid (SP) や IdP がサーバ側で保持している。これは Cookie を書き換えたなりすましを防ぐための意図的な設計 (IdP の Sealer 鍵 §9.2) がこの暗号化を支えている)。したがって「Cookie の中身を読んで認証情報を確認する」ことはできない。確認は目的別に次の 3 手段を使う。

知りたいこと	手段	見えるもの
セッションが確立 / 引き継がれているか	ブラウザ開発者ツール	Cookie の存在と属性 (中身は読めない)
認証情報の中身がいつ・何の属性で	/Shibboleth.sso/Session	サーバ側保持する認証情報の実体
IdP が SP に何を渡したか 通言のもの	SAML-tracer	SAML アサーションの XML (NameID 等)

手段 1 : 開発者ツールで Cookie の存在を確認 F12 → Application (アプリケーション) タブ → 左メニュー Cookies → 対象ホストを選択 - _shibsession_... : SP のセッション Cookie (あるいは SP セッション確立)。 - _shibstate_... : SP が認証リクエスト時使う一時 Cookie。 - shib_idp_session : IdP のセッション

Cookie (例)はIdP `idp.plm-lab.local` に対するCookieなのでIdPのドメインを選択して確認する。Secure/HttpOnly/有効期限も確認できる (HttpOnlyによりJavaScriptからは読めない⇒XSS対策)。

手段2: `/Shibboleth.sso/Session` で認証情報の実体を確認 (推奨) `https://<SPのホスト名>/Shibboleth.sso/Session` を開く (ログインしたホスト名で開く。localhostで見えよ)。Cookieが指すサーバ側セッションの内容が表示される: Identity Provider/Authentication Time/Authentication Context Class/Attributes (mail = 01PLM01@plm-lab.local 等) /Session Expiration。「Cookieの中身を知りたい」という目的には、これが最も適切な手段。

手段3: SAML-tracerでSAMLアサーションを確認 ブラウザ拡張SAML-tracerを入れSSOログインを実行記録された通信のうちSAMLマークの付いたPOST (IdP - SPのACS)を選びSAMLタブでSAML ResponseのXMLを表示する。<NameID Format="...emailAddress">01PLM01@plm-lab.local</NameID>、<AuthnContextClassRef>、<Conditions NotBefore/NotOnOrAfter> (クロックスキュー関係、署名など)も直接確認できる。「IdPがSPに何を引き渡したか」を最も直接的に見られる。

セッション引き継ぎ (3) を実証したい場合: 手段2で `shibsession_...` が送信されていることを確認し、手段2で `Session` の `NameID` のセッションが見えることを確認するとCookieが引き渡し役であることが目に見える形で確認できる。

15.9 構築完了・本番 (PLM) への展開メモ

これで純 Windows 環境での emailAddress 形式識別子による Shibboleth SSO 連携の学習環境が完成しました (全11フェーズ)。本番のPLMへ展開する際の要点:

- 保護対象のwhoami.aspを実際のPLMアプリ `C:\inetpub\wwwroot` 配下に置き換える。PLMはWebサーバ層で確定したREMOTE_USER=01PLM01@plm-lab.localのようなメール形式を読み@の前を切り出して従来の識別番号に変換して自DBで照合する。この切り出し・変換はPLMアプリ Web.config の認証方式スイッチ+実装の責務でSP/IdPの構築範囲外。サイト全体保護のためPLMの各ページも認証必須とする。
- 顧客のIdP (Entra ID) と繋ぐ場合、今回自分で立てたテストIdPの代わりにEntra IDのフェデレーションメタデータをSPに登録する。Entraが出すNameID=emailAddress形式とPLMの突合キーが一致することを確認する (SP側attribute-map/REMOTE_USERの考え方は同じ)。Entra側で「個人番号@ドメイン」をNameIDに出す設定が前提。
- 組織展開の留意点 (本書中で既出): オフライン導入 (付録E)、LDAPのTLS化 (本番ではLDAPS/StartTLS)、秘匿情報の集約 (secrets.properties)、SP配置はC:\opt 推奨 (付録\$12.1)、ビルトインAdministratorの承認モード差異 (\$11.5)、評価版のrearm (付録A)。
- 別マシン (クライアントPC) からのアクセスを本番で使う。ホスト名解決を実IPに向け、rootCAをクライアントに頒布し、ファイアウォールで443/8443を許可する (付録編)。

以上でフェーズ1~11の全工程が完了です。本書は、同じ手順を組織の検証環境で再実行するための土台として利用できます。付録:A (評価版 rearm) / B (バックアップ・再発生検証) / C (時刻同期・参考) / D (トラブル・Windows 固有) / E (オフライン導入)。

16. 発展編：LAN 上の別 PC・Hyper-V ホストからの接続

フェーズ11 までは「ゲスト Windows 上のブラウザ」で検証できた。ここでは **Hyper-V** ホストマシンや LAN 上の別 PC のブラウザから同じ検証環境の SSO を利用できるようにする。

実機確認済み：構築のまま **Hyper-V** ホストマシンのブラウザからの **SSO** が想定どおり動作することを確認済み。ゲストの hosts / 到達性を整えれば、SSO の設定変更なしにホストからログインできる。おまけでフェーズ6 \$10.3 で 8080 を localhost 限定してても外部からの SSO に影響しないことも確認できた。SSO の経路は 443 → 8443 で 8080 は含まないため。

重要な前提（構築は変更しない）：この発展編に必要なのはネットワーク到達性とサーバ証明書の信頼の設定だけであり、**IdP / SP / LDAP (SSO)** の構築内容は一切変更しない。Shibboleth は「アクセス元の IP」ではなく「ホスト名」で判断するため接続が変わっても URL のホスト名が sp.plm-lab.local / idp.plm-lab.local である限り entityID・メタデータ・NameID・REMOTE_USER・<Site name="sp.plm-lab.local"> 判定はそのまま機能する。しかたして本編で触るのは以下の2観点のみ。

16.1 観点の整理（2つだけ）

観点	内容	本編での扱い
① HTTPS サーバ証明書の種類	接続元ブラウザが内部 CA (PLM-Lab Root CA) 発行のサーバ証明書を信頼するか	rootCA を信頼する（警告なし）か、警告を容れる（配布なし）かを選ぶ (16.2)
② ネットワーク通信何	接続元 PC から仮想マシンの 443 / 8443 ままで通信が届くか	箱解決・仮想スイッチ・ファイアウォールを設定 (16.3)

① ② の両方は「そもそも通信届く」の話。両方を満たして初めて別 PC から SSO が使える。

16.2 観点①：HTTPS サーバ証明書の信頼

本環境のサーバ証明書は内部 CA (PLM-Lab Root CA) で発行した自己署名系のため、接続元 PC はこの CA を信頼してないと証明書警告を出す。対処は2通り。

- 方法 X（配布あり・警告なし・推奨度：本番向き）：接続元 PC の「信頼されたルート証明機関」に rootCA.crt (C:\lab\ca\rootCA.crt) を配布・登録する。以後、警告は出ない。
 - 手動: rootCA.crt を接続元 PC にコピーし、管理者 PowerShell で

```
Import-Certificate -FilePath "C:\path\to\rootCA.crt" -CertStoreLocation Cert:\LocalMachine\Root
```
 - Active Directory ドメイン環境なら、組織の CA をグループポリシーで配布する運用が一般的（ドメイン参加 PC は配布済みのことか多い）。

- 方法 Y (配布なし・警告あり・推奨度：検証/クローズド網向き) :rootCA を配布せず、ブラウザの証明書警告を「詳細設定 - このサイトに進む (安全でない)」で通過する。
 - クロスネットワークで接続 PC への配布を避けたい場合の割切。
 - 留意点 :SAML の往復 (SP←IdP) で sp.plm-lab.local と idp.plm-lab.local の両方のホストで警告が出るため 初回 2 回警告を通過する必要がある。警告状態は中間者攻撃できないことを意味するため、クローズド網に限った運用とする。

本書では、要件 (クローズド網・配布を避けたい) に応じて方法 Y (警告許容) で利用可能。警告を出さない本番寄せの場合方法 X rootCA 配布 または組織 CA の利用)。

16.3 観点②：ネットワーク通信許可

「ゲスト内から 127.0.0.1」で完結していた通信を、「外部 PC から仮想マシンの実 IP へ」届くようにする。次の 3 点を設定する。

(1) Hyper-V 仮想スイッチ (疎通の土台)

接続から仮想マシンの実 IP に到達できる必要がある。仮想マシンの接続を以下のいずれにする (Hyper-V マネージャー - 仮想マシンの設定 - ネットワークアダプタ、および仮想スイッチマネージャー)。

- 外部 (External) スイッチ :仮想マシンが物理 LAN に直接つかり、LAN 上の別 PC から実 IP で到達できる LAN の別 PC から使うならこれが素直。
- 内部 (Internal) スイッチ :ホストと仮想マシンの間のみ疎通 Hyper-V ホストからのみアクセスするだけ。
- 既定の Default Switch (NAT) :ホストゲストの特定ポート到達制御が厳しくあり、別 PC から基本届かない。別 PC 利用時は必ずスイッチを変更する。

(2) 仮想マシンの実 IP を確認し、接続元の hosts に登録

仮想マシン内実 IP を確認:

ipconfig | Select-String "IPv4"

接続 PC (ホスト or 別 PC) の hosts C:\Windows\System32\drivers\etc\hosts、管理者権限にその実 IP で登録する (127.0.0.1 でない)。

```
192.168.x.x sp.plm-lab.local
192.168.x.x idp.plm-lab.local
```

192.168.x.x は仮想マシンの実 IP。SP・IdP は同一の仮想マシンなので両ホスト名とも同実 IP を指す。

(3) 仮想マシンのファイアウォールで 443 / 8443 の受信を許可

ゲスト内 hosts から 127.0.0.1 だけ間通信内部で完結。受講者意識と動いていたが、外部からのアクセスは受信許可必要。仮想マシン内管理者 PowerShell で:

```
New-NetFirewallRule -DisplayName "Allow HTTPS 443 (SP)" -Direction Inbound -Protocol TCP -LocalPort 443 -Action Allow
```

```
New-NetFirewallRule -DisplayName "Allow HTTPS 8443 (IdP)" -Direction Inbound -Protocol TCP -LocalPort 8443 -Action Allow
```

疎通確認 (接続PCのPowerShell) :

```
Test-NetConnection -ComputerName 192.168.x.x -Port 443
```

```
Test-NetConnection -ComputerName 192.168.x.x -Port 8443
```

両方のTcpTestSucceeded がTrueなら正常OK。Falseの場合は(1) 仮想スイッチの種類 (3) ファイアウォール 同一ネットワークに属しているか (サブネット) を見直す。

16.4 接続確認 (別PC/ホストから)

接続PCのブラウザ (プライベートウィンドウ推奨) で:

1. https://sp.plm-lab.local/whoami.asp を開く
2. (方々 の場合) sp.plm-lab.local の証明書警告を通過
3. IdP ログイン画面 (https://idp.plm-lab.local:8443/...) へ遷移 (方々 の場合) idp.plm-lab.local の証明書警告も通過
4. 01PLM01 / 01PLM01 でログイン
5. whoami.asp (方々) REMOTE_USER = [01PLM01@plm-lab.local]

#	確認内容	期待結果
1	疎通	Test-NetConnection で443/8443 がTrue
2	名前解決	接続hosts でsp/idp が仮想マシンの実IPに解決
3	証明	方々 なら警告なし/方々 なら警告を通過してアクセス可
4	SSO	別PC/ホストのブラウザでもログインできREMOTE_USER に識別が入る

SSO の設定 (IdP/SP) 両方 Test-NetConnection がFalse なら 16.2② 16.3。

16.5 本番 (組織内ネットワーク) での考え方

- 本番はクロスドメインネットワークで 接続PC への個別配布を避けたい場合、**CA (AD CS 等)** でサーバ証明書を発行すればドメイン参加PCにはCA が配布されるため追加配布なしで警告も出ない (方々 の組織版)。内部自己署名CA を使った場合は各クライアントで信頼 配布 or 警告内容 が必要なる。
- ネットワーク面は 本番では仮想スイッチではなく実際のサーバLAN 構成なるが、接続サーバの443/8443 が名前解決・経路・ファイアウォールで通れることという要件は同じ。
- IdP を顧客のEntra ID に差し替える場合も、SP(IIS)側のホスト名・到達性の考え方は本編同じ。

16.6 本番でのサーバ証明書の用意（顧客システム／ネットワーク管理者向け）

本書の検証環境では、内部自己署名CA（PLM-Lab Root CA）でサーバ証明書を発行したが、これは学習・検証専用である。実業務（社内クロスネットワークでの本番運用）では、組織の正式証明書基盤に置き換えることを前提とする。以下は般的設計の考え方であり、最終的な方式は顧客のセキュリティポリシー・既存PKIに依存するため顧客システム／ネットワーク管理者・セキュリティ部門の判断が必要である。

想定されるサーバ証明書の用意方法（可能性の高い順）

パターン	内容	クライアント配布／警告	位置づけ
A：社内CA（AD CS）で発行	Active Directory 上の社内証明局（AD CS）から実FQDNのサーバ証明書を発行	社内CAのルートはGPOでドメイン参加PCへ配布済のため配布不要・警告なし	最有力・推奨。 ADドメイン環境でも標準的
B：パブリックCAの証明書	DigiCert／Sectigo／Let's Encrypt等の公的CAが発行	ブラウザが最初から信頼するCAのため配布不要・警告なし	FQDNが公的に登録・解決できる名前の場合に限る （例：local等の内部専用名には発行不可）
C：自己署名＋配布	自己署名簡易内部CAを作製し、ルートを実クライアントへ配布（GPO or 手動）	各クライアントへ配布が必要	CA基盤が無い小規模・一時的用途向け。更新・失効管理が手動で運用コスト高

大企業・中堅企業のクローズド業務システムではパターンA（AD CS）が最も多い。まず「社内CA（AD CS）を運用しているか」を確認するのが第一歩。

見極めのための確認事項：①Active Directory 上のAD CS 社内CAの有無、②FQDN

実業務へのアドバイス（証明書準備）

- 学習用の自己署名CAは本番に持ち込まない。実業務では組織の正式証明書基盤で発行した証明書を置き換える。差し替えるのはサーバ証明書（a）だけでSP/IdPの設定（entityID・メタデータ・NameID等）は変わらない。
- 社内CA（AD CS）があれば最優先で利用。ルート証明書はGPO配布済のためクライアントへの追加配布も証明警告も不要となる（本番で第1選択）。
- 証明書の名前（CN／SAN）を実FQDNで用意。本番で実際に使ったホスト名をSANに含めて発行するSANとアクセスURLのホスト名が一致しないと警告が出るのは本番でも同じ。IIS（SP）とTomcat（IdP）それぞれのホスト名分が必要。

- 有効期限と更新・失効の運用を最初に決める。証明書切れるとサービス停止する。更新期のアート、更新順 (IIS 側の証明書差し替えと Tomcat 側の PFX 差し替えの両方)、失効対応を導入時に整理する。
- 鍵の管理を引き締める。サーバ秘密鍵・PFX (は Web 公開域 C:\inetpub 配下) に置かない、アクセス権を絞る。パスワードを証明書の既定値 (changeit 等) から変更する。
- TLS 証明書 (a) と SAML 署名・暗号化証明書 (b) は別物として整理してやる (a) は信託のお組織 CA / ブロック CA で用意するが (b) は Shibboleth から生成しメタデータ交換で取得するもので CA 署名不要 §6.3 参照。顧客管理者が証明書を振り出して困らなよう 2 種類に分けて使用する。

ネットワーク面を離せば本番では仮想スイッチではなく実サーバ実 LAN 構成だが、接続元サーバの 443/8443 から暗号化・経路・ファイアウォールを踏越えることという要件は §16.3 と同じ。

17. 発展編：組織内の複数 PLM 環境の SSO 対応（並行運用の設計メモ）

位置づけ：本節は、同一の IIS 上で複数の PLM テスト環境 (別ポート) を運用してるケースを従来認証 (HTTP) を維持したまま、SSO (HTTPS) を新ポートで追加して並行運用するための設計メモである。実機検証の想定構成。実稼運用は組織のポリシー・PLM 実装に依存するためシステム管理者・PLM 開発担当と相談して確定すること。実構築・検証後に本節を確定版へ更新する想定。

17.1 背景と方針

同一の Web アプリケーションサーバ (IIS) 上で複数の PLM 環境別ポートで提供してるとする (例: 開発1=80、検証=9012、開発2=9013。いずれも HTTP・従来 Cookie 認証・ユーザは 01PLM01/01PLM02 等を共有)。顧客側でも当面は「一部ユーザは SSO、他は従来の HTTP 独自認証」という並行運用が予定されるため、自組織の環境両方式に対応できるようにしておく。

方針：各環境について、従来の HTTP ポートはそのまま維持し、SSO 用に HTTPS の別ポートを新設する。同じ物理フォルダ (同じ Web アプリ) を両サイトから提供し、認証方式は `appSettings` の `AuthMode` を IIS のサイト単位設定で上書きして切り替える (図 17.3(d))。SSO 用ポートは Shibboleth SP で保護し、HTTP ポートは保護しない (従来認証のまま)。

17.2 構成イメージ

SP は既存ホスト名 `plmdev.plm-lab.local` に統一し、ポート番号で環境・認証方式を識別する。IdP は別ホスト名 `idp.plm-lab.local` に分ける (ホスト名付けの根拠は 17.3(f))。

物理 1 台 (開発サーバ / 同一マシンに 2 つのホスト名を割り当て)

└─ `plmdev.plm-lab.local` (既存=SP / PLM。ポートで環境・認証方式を識別)

└─ 【従来認証 (HTTP・そのまま維持 / SP 保護しない)】

└─ (1) `http://plmdev.plm-lab.local/` → PLM 開発 1 (Cookie 認証・80)

└─ (2) `http://plmdev.plm-lab.local:9012/` → PLM 検証 (Cookie 認証)

└─ (3) `http://plmdev.plm-lab.local:9013/` → PLM 開発 2 (Cookie 認証)

└─ 【SSO (HTTPS・新設 / 同一 IIS 上で SP が保護)】

└─ (1') `https://plmdev.plm-lab.local:4443/` → PLM 開発 1 (SSO) └

└─ (2') `https://plmdev.plm-lab.local:9112/` → PLM 検証 (SSO) └ 同一 entityID (方式 1)

れるため「サイトごとのWeb.configを置く」ことはできない。代わりに IIS の「アプリケーション設定」でサイト単位に **appSettings** を上書きする。同じWeb.configのまま、サイトごと違う値を持つ。

```
<!-- Web.config (共有) : 既定は従来認証 -->
<appSettings>
  <add key="AuthMode" value="Legacy" />
</appSettings>
```

- IIS マネージャー → **SSO** サイト (**HTTPS**) を選択 → 「アプリケーション設定」 → **AuthMode** を **SSO** に設定 (書き)。
- 結果: HTTP サイト経由 = Legacy (Web.config の既定値) / HTTPS サイト経由 = SSO (IIS で書き)。

共通認証処理 (既存の「Cookie から識別番号を取得する共通関数」) に分岐を追加する。認証
×××××××××××××××××××××××× 1箇所済み、子画面は無改修:

```
Public Shared Function GetCurrentUserId() As String
  Dim authMode = ConfigurationManager.AppSettings("AuthMode")
  If authMode = "SSO" Then
    'SSO : SP が確定した REMOTE_USER から取得
    Dim remoteUser = HttpContext.Current.Request.ServerVariables("REMOTE_USER")
    If String.IsNullOrEmpty(remoteUser) Then
      Throw New ApplicationException("SSO モードだが REMOTE_USER が空 (SP の設定漏れ)")
    End If
    Return remoteUser.Split("@")(0) '01PLM01@plm-lab.local → 01PLM01
  Else
    Return GetUserIdFromCookie() '従来 : Cookie から (既存処理)
  End If
End Function
```

⚠ **AuthMode=SSO** なのに **REMOTE_USER** が空なら例外にする (黙って従来認証にフォールバックすると、認証を迂回できてしまう)。AuthMode=Legacy のサイトでは SP 保護のため **REMOTE_USER** は常に空で、これは正常。

<location> は使えない : ASP.NET の **<location path="...">** はパス単位の設定であり、サイト (ポート) 単位ではないため、同じシステムを別サイトから提供する構成では使えない。上記の IIS のアプリケーション設定で上書き」が正解。

(e) 同一ホスト名・証明書は 1 枚で共有 (組織開発・検証環境は SP 側既存の同一ホスト名 plmdev.plm-lab.local) のポートとする。この場合 SP のサーバ証明書は **SAN** にそのホスト名を含む 1 枚で全 **SSO** ポートを共有できる (§16.6)。方式 (全ポート同一 entityID/Application) なので SP のセッションはポート間で共有され、「1 回ログインで 3 環境に入る」挙動になる。並行運用の検証用途で有効。

(f) ホスト名設計 : **SP** は既存ホスト名に統一・**IdP** は別ホスト名に分離 (既に PLM の開発・検証環境に決まったホスト名 (例 plmdev.plm-lab.local) がある場合) 次の方針推奨:

- **SP (PLM を保護する側)** は既存ホスト名 **plmdev.plm-lab.local** に統一し、ポート番号で環境 (開発 1/検証/開発 2) と認証方式 (**HTTP=従来** / **HTTPS=SSO**) を識別する。既存の解決・運用をそのままに証明書 1 枚 (SAN=plmdev) で済み \$17 の並運用に合わせる。
- **IdP (認証する側)** は別ホスト名 **idp.plm-lab.local** に分ける。plmdev=PLM/SP、idp=IdP (IdP か Entra ID (別ホスト/クラウド) になるため検証でも IdP と SP と IdP を同名すると TLS 証明書・SAML メタデータ (entityID) がホスト名ベースで紛れく取壊事故起きやすい)。
- 物理的には 1 台のマシンに両ホスト名 (**plmdev.plm-lab.local** / **idp.plm-lab.local**) を割り当てる (同一 IP)。今回の学習環境で 1 台に sp/idp の 2 名を割り当てての同考方 DN S (または hosts) で両ホスト名を同一 IP に向け証明書は SP 用 (plmdev) と IdP 用 (idp) を別にと 1 枚ずつ用意する。

「すべて plmdev.plm-lab.local に統一してポートだけで区識別」も技術的には可能だが IdP も同名すると役割混ざり、証明書・メタデータ紛れくなくなるため検証 IdP だけは別ホスト名に分けるのがよい。

17.4 構築の流れ (想定手順・要実機検証)

1. IIS に SSO 用サイト (1')(2')(3')) を追加、各ポート (例 4443/9112/9113) に **HTTPS** バインドを設定 (証明書同一ホスト名 1 枚共有 \$11.4 と同要項)。
2. 同じ物理フォルダを SSO 用サイトから参照させる (アプリコピ要)。IIS のアプリケーション設定で SSO サイトの AuthMode を SSO に書き (17.3(d))。従来 HTTP サイトはそのまま。
3. shibboleth2.xml の <ISAPI> に SSO 用ポートの <Site> を追加 (17.3(b))。<RequestMapper> で当該サイトを requireSession="true"。方式 1 のため entityID は 1 つ。
4. shibd.exe -check %sbin64/sbin) - Restart-Service shibd_Default - iisreset。
5. SP メタデータ 1 つを IdP に登録 (\$13.4 と同じ)。IdP・LDAP は共有のまま。
6. 各 SSO 用ポートにブラウザでアクセス - IdP ログイン - 各 PLM 環境に SSO で入れることを確認 (従来 HTTP ポートは従来お馴染みことも確認)。

17.5 留意点

- **HTTPS 化は SSO の前提** : SP のセッション Cookie は Secure 属性付き、SAML の往復も HTTP S 前提。SSO 用ポートは必ず HTTPS にする (HTTP のままでは SSO は成立しない)。
- セッション共有の範囲 : 方式 1 では 3 環境同一 Application のため SP セッションが共有される 1 回ログインで全 SSO 環境に入る。環境ごとセッションを属性解放分ける必要が出たら方式 2 の <ApplicationOverride> で環境別 entityID・別メタデータへ発展させる。
- 本番 (顧客) との整合 : 顧客の「一部 SSO・一部従来認証」の並運用と同構造。顧客本番では IdP か Entra ID に替わるが SP 側 (ポート単位保護・REMOTE_USER 受渡し) の考方は同じ。
- 本節は設計メモ : 実機の構築・検証を経てポート設計・<ISAPI> / <RequestMapper> の具体値・証明書 SAN・AuthMode 切り替えの実装を確定し、本節を更新すること。

17.6 開発・ビルド環境の構成 (Visual Studio / ローカル IIS)

既存の PLM 開発環境 (NET Framework 4.8 / Visual Studio 2019 / VB.NET、ローカル IIS の Default Web Site (80/HTTP) にプロジェクトフォルダを既定ディレクトリでマッピング。スタートページ MainWeb.aspx から子画面 Form.aspx へ遷移) を SSO 対応する形に構成。

(a) 推奨構成：プロジェクトは1つのまま、同じ物理フォルダを2サイトから提供

【Visual Studio】PLMApp.vbproj (1つのまま)

- └ MainWeb.aspx ← 従来 (社内テスト用・Cookie 認証前提)
- └ MainWebSso.aspx ← ★新規追加 (SSO 版スタートページ)
- └ Form.aspx … ← 子画面 (★共有・無改修。コピーしない)
- └ AuthHelper.vb ← ★共通認証処理 (ここだけ改修：AuthMode で分岐)
- └ Web.config ← <appSettings> に AuthMode (既定 Legacy)

【ローカル IIS】同じ物理フォルダを2サイトから参照

Default Web Site (80/HTTP) → 仮想ディレクトリ /PLMApp → C:\dev\PLMApp\

└ アプリケーション設定：AuthMode = Legacy (既定のまま)

└ SP 保護：なし (shibboleth2.xml の <Site> に登録しない)

PLMApp-SSO (443/HTTPS) → 仮想ディレクトリ /PLMApp → C:\dev\PLMApp\ (★同じフォルダ)

└ アプリケーション設定：AuthMode = SSO (IIS で上書き)

└ SP 保護：あり (<Site> に登録)

要点 :IIS は複数サイトが同じ物理フォルダを指してよい。これにより、ソースは箇所・プロジェクトも1つのまま、443 サイト経由なら「メイン画面も子画面もすべて SP に保護される」(サイト全体保護 §12.3)。認証処理が共通関数に集約されていれば、改修は共通関数 1 箇所だけで子画面 (Form.aspx) は無改修で両対応できる。

(b) 「SSO 用に別プロジェクトを作る」案が成立しない理由

SSO 用新プロジェクト (例: PLMSSOApp.vbproj) を作り、そのスタートページから既存 PLMApp の子画面を呼び出すという案が成立しない。理由:

1. **SP の保護が子画面に及ばない (致命的)** : SP の保護はサイト単位。MainWebSso.aspx (443・SP 保護あり) から遷移しても Form.aspx が 80 サイトの既存アプリにある限保護対象外で REMOTE_USER は空になる。子画面直接 URL を打てば認証なしでアクセスできてしまう。
2. **ASP.NET セッションが共有されない** : 別プロジェクト = IIS 上の別アプリケーションのため Session 変数が別物なる。同アプリ内画面遷移といえども既存前提が崩れる。
3. 実質的にソース共有になっていない : 「別アプリの画面に遷移しているだけ」で、保護されたコードを使っていることになる。

保護すべき画面 (メイン + 子画面) は、同じ保護されたサイト内に存在する必要がある、というのが設計上の制約

(c) デバッグの進め方

- 従来認証のデバッグ : 既存のまま http://localhost/PLMApp/MainWeb.aspx (80) で Visual Studio のデバッグ実行 (AuthMode = Legacy)。これまでの開発フローを変えなくてよい。
- **SSO のデバッグ** : https://localhost/PLMApp/MainWebSso.aspx (443) にアクセス - IdP ログイン - Visual Studio の「プロセスにアタッチ」(w3wp.exe) でデバッグ。

- **SP/IdP** として **SSO** ロジックだけ確認したい場合（モック）:REMOTE_USER を定値で利用すれば **IdP** に接続せずに **SSO** 版の画面遷移・@ 切り出し・認可判定をローカルでデバッグできる

```
Dim remoteUser = HttpContext.Current.Request.ServerVariables("REMOTE_USER")
#If DEBUG Then
    If String.IsNullOrEmpty(remoteUser) Then
        remoteUser = ConfigurationManager.AppSettings("DebugRemoteUser") '例: 01PLM01
        @plm-lab.local
    End If
#End If
```

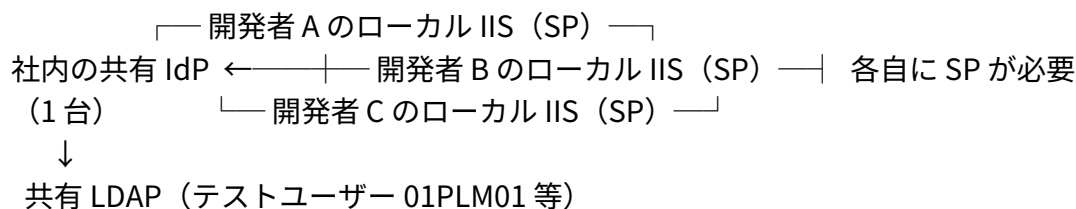
(d) 各開発者のローカル IIS に SP は必要か → 必要

SP は「保護する Web サーバに常駐するもの」^① Shibboleth サービス+IIS ネイティブモジュールであり、REMOTE_USER を確定して IIS に渡すのは SP。したがって、各開発者のローカル IIS で **SSO** を動かすなら、その IIS ごとに **SP** の導入・設定が必要^② SP を 1 か所集約して共有することはできない。開発者ごとに必要なるもの:

項目	開発者と異なるか
SP のインストール (MSI・IIS モジュール構成)	手順は同じ
ホスト名・entityID (例 dev-a.plm-lab.local / https://dev-a.plm-lab.local/shibboleth)	開発者ごとに異なる
サーバ証明書 (a) (そのホスト名の TLS 証明書 SP 鍵(b)③keygen)	開発者ごとに必要 開発者ごと生成
SP メタデータの IdP への登録	開発者の数だけ登録が必要
<ISAPI> の <Site> /attribute-map/REMOTE_USER	考え方は同じ

(e) IdP は社内に 1 か所を共有

IdP は社内に 1 台だけ立て、全開発者の **SP** がそれを共有する^④ 1 IdP : 多 SP = 標準構成。全員が同じテストユーザー (01PLM01 等) で認証でき、本番構成とも構造が一致する。IdP 側は各開発者の **SP** メタデータを登録し (metadata-providers.xml)、属性絞り込み (attribute-filter.xml) も各 SP 分を用意する (entityID をまとめて何冊か書ける)。⑤



本書の方針（開発メンバーが少数のため）: 各開発者のローカル IIS に SP を導入する構成 (上記) を採る。開発者が増えて運用・証明書発行・メタデータ登録の期間が問題となる場合は共有の「SSO 検

証サーバ」を1台だけ立て、日常のデバッグは各自ローカル（80/HTTP + REMOTE_USER のモック）で行い、SSOの実動作は検証サーバで確認する構成切り替える余地ある SP は 1 台のみで運用が利便的になる。

付録 A：評価版の rearm 運用

slmgr /dlv # 残り日数・rearm 回数の確認
slmgr /rearm # 延長（実行後は再起動）

仮想マシンを削除すると全スタックが失われます。期限管理・削除・作直ではなく rearm で行ってください、フェーズ1の前提。

付録 B：バックアップと再現性検証

付録 B-1：参照用ファイルバックアップ一覧（再構築時の“答え合わせ”用）

位置づけ：本環境の目的は「消滅した素の状態から再度完成までできる」の検証。またバックアップは重複せず前回どう設定したかを確認するための“答え合わせ用”にテキスト設定ファイルを絞る。

IdP (C:\opt\shibboleth-idp\) : conf\ldap.properties、conf\attribute-resolver.xml、conf\attribute-filter.xml、conf\saml-nameid.xml、conf\relying-party.xml、conf\metadata-provider.xml、credentials\secrets.properties、metadata\idp-metadata.xml、metadata\sp-metadata.xml。**Tomcat** : C:\opt\tomcat\conf\server.xml、conf\Catalina\localhost\idp.xml。**SP (C:\opt\shibboleth-sp\etc\shibboleth\)** : shibboleth2.xml、attribute-map.xml、idp-metadata.xml。**IIS** : C:\inetpub\wwwroot\whoami.asp。**LDAP (ApacheDS)** : 投入した LDIF C:\lab\ldap-users.ldif 等。ApacheDS のデータ本体は C:\Program Files (x86)\ApacheDS\instances\default\partitions\。証明書（値の照合用） : C:\lab\ca\一式 {rootCA/idp/sp の crt・key・pfx}。再構築では作り直すため参照用。

収集例（PowerShell） : New-Item -ItemType Directory C:\lab\ref -Force; Copy-Item C:\opt\shibboleth-idp\conf*. * C:\lab\ref\idp-conf\ -Recurse のように用途に集めて zip 化しておく、答え合わせに使える。

 バックアップは必ず **IdP** の外へ : conf\credentials\ 配下に拡張子 **.properties** のままコピーを残さないこと（IdP が読み込み、編集の既定値競合して認証失敗する）。バックアップは C:\lab\ref\ C:\lab\idp-backup\ など **IdP** の外に置くか、ldap.properties.orig のように拡張子を変える（§9.4 付録D）。

付録 B-2：再現性検証のためのチェックポイント運用

目的 : 素のソフトウェアから消滅してから構築し、REMOTE_USER=[01PLM01@plm-lab.local] まで達できることを検証する。完成環境は保存して時の記録だけにする。

推奨手順：

1. ゲストWindows をシャットダウン（電源オフ）。

2. (任意) Set-VM -Name "<VM 名>" -CheckpointType Standard。本構成は入れ子仮想化を使わないため標準 (Standard) チェックポイントで可。停止状態の取得が最も安全
3. 現在完成状態のチェックポイントを取得 - ツールに表示されたことを確認
4. 作業前チェックポイントを「適用」適用の追加作成は不要。
5. 適用後時刻を確認 (付録)。手順書に沿って再構築。
6. 検証完了後、保存した完成状態チェックポイントを削除 (ディスク解放)。

パラメータ名: VM 自体を操作する系 (Set-VM/Checkpoint-VM 等) は Name、VM 構成要素を操作する系 (Set-VMProcessor/Set-VMMemory) は VMName。取捨を避けるには Get-VM "<VM 名>" | Set-VM -CheckpointType Standard。

18. 発展編：実サーバ (Windows Server 2016 等) での構築時の追加考慮事項

学習環境 (Windows 11 / Hyper-V) と実稼働サーバ (Windows Server 2016 等) とでは既定設定や導入済みコンポーネントが異なる。実サーバで構築した際に判明した追加の考慮事項をまとめる。実機対応・確認済み。フェーズ本編併せて参照。

18.1 TLS 1.2 の有効化 (Windows Server 2016)

Windows 11 では TLS 1.2 が既定で有効だが Windows Server 2016 では明示的な有効化が必要な場合がある。IIS (SP) は OS の TLS スタック (SCHANNEL) を使いため、無効と HTTPS 接続成立しない。管理者 PowerShell で SCHANNEL のレジストリを設定する。

```
$base = "HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\SCHANNEL\Protocols\TLS 1.2"
foreach ($role in "Server","Client") {
    New-Item -Path "$base\$role" -Force | Out-Null
    New-ItemProperty -Path "$base\$role" -Name "Enabled" -Value 1 -PropertyType DWORD -Force | Out-Null
    New-ItemProperty -Path "$base\$role" -Name "DisabledByDefault" -Value 0 -PropertyType DWORD -Force | Out-Null
}
```

- 設定後は OS の再起動が必要。SCHANNEL の変更は再起動で反映。
- Server と Client の両方に Enabled=1/DisabledByDefault=0 を入れる。片方だけ・再適用漏れだと IIS が TLS を通せず HTTPS が遮断される。
- Tomcat (IdP・8443) は Java 独自の TLS スタックを使いため、この設定の影響を受けない。IIS (443) だけが繋がらない場合、OS の TLS 設定を疑うと切り分けは使える。
- IIS/.NET が絡む場合、SchUseStrongCrypto (HKLM:\...\Microsoft\.NETFramework\v4.0.30319 および WOW6432Node 側=1) も併せて必要になることがある。

18.2 Tomcat サービス導入後、OS シャットダウンが完了しない

Tomcat をWindows サービス化後 OS シャットオン時に「電源を切らないでください」が表示され続け、完了しない現象が起きることがある (shutdown -s -t 0 -f の実行で停止できる)。OS からTomcat サービスの停止完了を待っているのが原因

- 対処 (実機で解消) : サービスの停止タイムアウトを短く設定する (実機では20秒に設定して解消)。Tomcat サービス (Commons Daemon procrun/tomcatXw.exe) のShutdown タイムアウト、またはHKLM:\SYSTEM\CurrentControlSet\ControlのWaitToKillServiceTimeout (ミリ秒・全サービス共通のため慎重) を調整する
- 切り分け : Stop-Service <Tomcat サービス名> で停止してからOS をシャットダウンし、スムーズに落ちるが確認する。停止してスムーズにシャットダウン時のタイムアウト問題。停止してもシング=Tomcat/IdPの停止処理自体の問題
- IdP はメタデータ更新等のバックグラウンドスレッドを持つため、停止時に時間がかかることがある。catalina.*.log/ldp-process.log にシャットダウン時の手がかりが無く、確認する

18.3 証明書 : OpenSSL 3.x では PKCS#12 に -legacy が必要

実サーバにOpenSSLが無場合、別PCで証明書を作ってコピーする。あるいはサーバにGit for Windowsを入れて作る (サーバ自身で作るのが確実。\$18.4)。ここで **OpenSSL 3.x** で作った **PFX (PKCS#12)** が **Windows** の証明書インポートで「パスワードが不一致」エラーになることがある

- 原因 : OpenSSL 3.x はPKCS#12の既定暗号化方式を新しいものに変更した。Windowsの証明書インポートは従来方式RC2等を期待するため、方式の食い違いが「パスワード不一致」という紛れもないエラーで現れる (実際はパスワードの問題ではない)。
- 対処 (実機で解消) : PFXの作成時・確認時の両方で**legacy** オプションを付ける

作成 (従来方式でPFXを書き出す)

```
openssl pkcs12 -legacy -export -out sp.pfx -inkey sp.key -in sp.crt -certfile rootCA.crt -passout pass:changeit
```

確認 (パスワード・中身の検証)

```
openssl pkcs12 -legacy -info -in sp.pfx -noout -passin pass:changeit
```

- 学習環境 (Windows 11 + Git for Windows のopenssl) で問題が出ないのはOpenSSLのバージョン差による。実サーバのopensslが3.xなら**-legacy**を付ける。
- 「パスワード不一致」が出るとき、まずopenssl pkcs12 -info -in sp.pfx -noout -passin pass:changeit (必要ならlegacy付き) で本当にパスワードの問題か暗号化方式の問題かを切り分ける

18.4 証明書はサーバ自身で作るのが確実

別PCで証明書を作ってコピーすると、作成時のパスワードの食い違いやファイルのコピー事故 (改行コード変換等) が起きやす。サーバ自身に **Git for Windows (openssl 同梱)** を導入し、本書 \$6 の **openssl** 手順をそのサーバ上で実行して証明書一式を作るのが最良

- パスワードをその場でchangeitに統一でき、以降のインポートも通る

- ・ 学習環境の\$6 と同じ手順をサッ止で行うだけで、手順の整合も取れる

18.5 既存サイトがある環境：SSO 用サイトのサイト ID に注意（重要）

実際に既に **Default Web Site (ID=1)** がある環境で SSO 用別サイトを追加すると、そのサイトの ID は **1** 以外（例：2）になる。学習環境の本編は Default Web Site (ID=1) を前提に `<Site id="1">` としていたが、別サイトを追加した場合は、そのサイトの実際の ID に合わせる必要がある。

- ・ サイト ID の確認：

```
Import-Module WebAdministration
Get-Website | Select-Object Name, ID, State
```

追加した SSO 用サイト (例 ShibbolethTest) の ID を確認する (例では 2)。

- ・ **shibboleth2.xml** の `<ISAPI>` を、そのサイト ID に合わせる：

```
<ISAPI normalizeRequest="true" safeHeaderNames="true">
  <Site id="2" name="sp.plm-lab.local" scheme="https" port="443"/>
</ISAPI>
```

こゝを id="1" のまますると、SP が実際のサイト (ID=2) を保護せず、**Shibboleth.sso/Status** も **404** のままになる。動かすサイトを作った場合の最悪の落とし穴

- ・ `<RequestMapper>` の `<Host name="sp.plm-lab.local" ...>` も追加サイトのホスト名に合わせる

18.6 ホスト名バインドと localhost アクセスの 404（正常な挙動）

SSO 用サイトの 443 ノードを ***:443:sp.plm-lab.local** (ホスト名固定) にすると `https://localhost/` は別の受け皿に振り分けられ **404** になる。かつ証明書 SAN が `sp.plm-lab.local` のため `localhost` では名前不一致で「セキュリティ保護なし」表示になる。これは構築の失敗ではなく、想定どおりの挙動。

- ・ テストは登録したホスト名 (**sp.plm-lab.local**) で行う。localhost は使えない。
- ・ **Shibboleth.sso/Status** は **shibboleth2.xml** で `<Site>` を登録して初めて有効になる。登録前 (フェーズ 12.2 段階) は 404 が正常。編集 `shibd -check · iisreset` の後 `https://sp.plm-lab.local/Shibboleth.sso/Status` で `<Status><OK/></Status>` を確認する。

切り分けの指針（今回の教訓）：「HTTPS が遮断される」ように見えても、① 接続セット (TCP/TLS) の 404 (サーバに到達しているがパスが無い／別サイトに振り分け) はまったく別の問題。`https://sp.plm-lab.local/whoami.asp` が鍵マーク付きて表示されるなら TLS 証明書・443 ノードバインド成立しており、残るはサイト振り分け (ホスト名・サイト ID) の問題、と切り分けられる。

付録 C：時刻の確認（参考）

本編は IdP・SP・LDAP がすべて同様の Windows 上にあるため SAML のクロックスキューは原理的に発生しない。ゲスト Windows の時刻合わせは十分 Hyper-V 統合サービス「時刻同期」が有効ならホストに通じる。オンライン環境では Wi

Windows Time(w32tm)がNTPに同期するスリープを使う運用で復帰時時刻がずれる場合はw32tm /resyncで再同期する

付録D：トラブルシュート (Windows 固有)

- **java / JAVA_HOME** が効かない : 環境変数を設定後、新しいセッションを開き直しか (§5.6)。echo %env:JAVA_HOME%。
- **Tomcat** サービスが起動しない : tomcat10w //ES//Tomcat10 のJava タブでJVM(jvm.dll)を確認 (§8.3)。logs\catalina.*.log。
- **8443** が **LISTEN** しない : server.xml のSSLHostConfig/Certificate の記述 conf\idp.pfx の有無ノスワード (§10)。
- **shibd.exe** の場所 : sbin64 またはsbin (§12.2)。
- **ApacheDS** に繋がらない : LDAP ポートは既定**10389** New Connection の既定表示389 (注意)。netstat -ano | findstr 10389。
- **Directory Studio** が起動しない : ApacheDirectoryStudio.exe を直接実行 zip 展開はスタートメニュー未登録。Java 未検出時はiniに-vmでJDKを指定 (§7.3)。
- ログイン時に **Pool is empty and connection creation failed** : conf/credentials 配下のバックアップコピー (dap - Copy.properties 等・拡張子がpropertiesのまま)を読み入れ、編集前の既定値が使われているのが最もバックアップはdap.properties.origのように拡張子を変えるかIdPの外置く。趣時のWARN ... Duplicate properties were detected が頻発 (§9.4・§13.7)。次はtrustCertificates/trustStoreの有効化を疑う。
- **/idp/profile/status** が **Access Denied** : hosts を実IPにした場合発生 status (既定でlocalhostの許可 (conf/access-control.xml)。異常でない status はhttp://localhost:8080/... で確認し、8443の確認はメタデータ (idp/shibboleth) で行う (§10.4)。
- **idp-process.log** に古い **WARN** : 追記のため最新のShibboleth IdP Version 行以降を見る (§15.5)。

付録E：オフライン導入

インターネットに出られない検用PCでは別のオンライン端末で以下を取得して持ち込む:

- Temurin 17(zip)、ApacheDS(exe)、Apache Directory Studio(zip)、Tomcat 10.1(zip)、Shibboleth IdP 5(zip)、Shibboleth SP(win64 msi)。
- **JSTL 2 jar**(jakarta.servlet.jsp.jstl-api-3.0.0.jar とjakarta.servlet.jsp.jstl-3.0.1.jar)。これらをC:\opt\shibboleth-idp\edit-webapp\WEB-INF\lib\に置いてからbuild.bat (§9.3)。
- 証明書検用PC上のGit Bash(openssl)で作成するためオンライン不要

使用したインストール版の実績は§3のリストを参照 (Temurin 17.0.19 / ApacheDS 2.0.0.AM27 / Directory Studio M17 / Tomcat 10.1.57 / IdP 5.2.3 / SP 3.5.2.3)。

[全工程完了] 本刊書 純Windows 版はフェーズ1〜4と付録A〜Eをもって完成です。WSLを使わずWindowsのみでShibboleth SSO 検証環境 (IS = SP、Tomcat = IdP、ApacheDS = LDAP) を構築し emailAddresses

s形式識別子をREMOTE_USERに載せるところまでを、実機検証に基づいて記載しています。

